FPGA-based Intrusion Detection for DoS Attack

Eoin Sheerin
ID: 20465956
eoin.sheerin.2021@mumail.ie

Department of Electronic Engineering
National University of Ireland, Maynooth
Ireland

Submitted to the Department of Electronic Engineering in partial fulfilment of the requirements for the Master of Electronic Engineering Meng(Hons) degree

Facilitator: Dr. Erivelton Nepomuceno

Word Count: (x)

Declaration

I have read and understood the Departmental policy on plagiarism.

I declare that this thesis is my own work and has not been submitted in any form for another degree or diploma at any university or other institution of tertiary education.

Information derived from the published or unpublished work of others has been acknowledged in the text and a list of references is given.

Student Signature:

Eoin Sheerin

Student Number:

20465956

Date:

05/2026

Acknowledgements

First and foremost, I would like to express my sincere gratitude to my supervisor, Dr. Erivelton Nepomuceno, for allowing me the opportunity to do this project, as well as your willingness to assist me in this matter.

I am also deeply grateful to Dr. John Dooley for providing the necessary hardware resources required for this research. His technical assistance and advice in overcoming implementation challenges were greatly appreciated.

Furthermore, I would like to thank the faculty and staff within the Department of Electronic Engineering at Maynooth University for their continued support and for providing a positive environment during my studies.

Finally, I would like to thank my family for their support and encouragement.

Abstract

This report presents the design and implementation of an FPGA-based Intrusion Detection System, which has the ability for real-time detection of Denial of Service attacks. This report outlines the core design challenges encountered during development of this project and also details the solutions implemented to address them. Testing was also conducted to evaluate the system's performance, with the resulting data proving its long-term reliability and operational stability.

Examples of existing intrusion detection approaches are discussed, along with relevant technical background, to provide context for the design choices made in this project. Particular focus is given to the advantages and limitations of implementing an IDS on FPGA hardware compared to traditional software-based solutions.

This project demonstrates the potential of hardware-accelerated intrusion detection, highlighting the benefits of low-latency processing and parallel execution for network security applications.

Contents

Declaration **i**

Acknowledgements **ii**

Abstract **iii**

Contents **iv**

List of Figures **vi**

1 Introduction **1**
1.1 Problem Statement and Project Solution . . . 2
1.2 Report Structure . . . 3

2 Technical Background **4**
2.1 Layers Of Networks . . . 4
2.1.1 Layer 3 . . . 4
2.1.2 Layer 4 . . . 4
2.1.3 Layer 7 . . . 4
2.2 Types Of DoS Attacks . . . 5
2.2.1 Application Layer Attacks . . . 5
2.2.2 Protocol Exploits . . . 6
2.2.3 Volumetric Attacks . . . 7
2.3 FPGA Architecture . . . 7
2.3.1 Xilinx Overview . . . 9
2.3.2 VC707 Platform . . . 9
2.4 MicroBlaze Processor . . . 10
2.5 AXI Interconnects . . . 11
2.6 Hardware-Software Co-Design . . . 11
2.7 Scapy and Traffic Generation . . . 11

3 Literature Review **12**
3.1 Prisma . . . 12

4 System Design and Development **13**
4.1 High Level System Overview . . . 13
4.1.1 System Architecture . . . 13
4.1.2 Data Flow Through the System . . . 14
4.1.3 Hardware-Software Partitioning . . . 15
4.1.4 Communication Interfaces and Interconnects . . . 16
4.1.5 Platform Integration and Physical Implementation . . . 16
4.2 Packet Replay and Data Source . . . 18
4.3 IDS Module . . . 20
4.3.1 Packet Parsing . . . 20
4.3.2 Feature Extraction and Signature Detection . . . 21
4.3.3 Sliding Window . . . 21
4.4 Detection and Alert Generation . . . 22
4.5 Microblaze . . . 22
4.5.1 System Configuration . . . 23
4.5.2 Packet Transmission and Status Polling . . . 23
4.5.3 Target Identification and Dynamic Tracking . . . 23
4.5.4 UART Reporting . . . 24
4.5.5 Software Operational Logic and Temporal Synchronisation . . . 24
4.6 Graphical User Interface . . . 26
4.7 Design considerations and challenges . . . 27
4.7.1 Feature Extraction and Traffic Monitoring . . . 27

5 Results and Testing **28**
5.1 Testing . . . 28
5.1.1 Packet Representation for a FPGA system . . . 28

5.1.2 Packet Dataset and Test Setup . . . 29
5.2 Results . . . 30
5.2.1 Threshold and Packet Rate Analysis . . . 30
5.2.2 High Packet Rate: `usleep(5000)` . . . 31
5.2.3 Baseline Packet Rate: `usleep(10000)` . . . 31
5.2.4 Low Packet Rate: `usleep(20000)` . . . 31
5.2.5 Discussion . . . 32
5.2.6 Syn Flood . . . 32
5.2.7 Mixed Packets . . . 33
5.2.8 ICMP Flood . . . 35
5.2.9 NULL Scan Detection . . . 35
5.2.10 XMAS Scan Detection . . . 36
5.2.11 UDP Flood Detection . . . 37
5.3 System Limitations . . . 37

6 Ethical Considerations **39**

7 Conclusions and Future Work **40**

List of Figures

1 DDoS attack diagram [10] . . . 1
2 Spike in traffic diagram [12] . . . 2
3 7 Layers of OSI [22] . . . 4
4 HTTP GET and POST Flood Attacks . . . 5
5 Diagram of a SYN Flood . . . 6
6 Volumetric DDoS Attack Architecture . . . 7
7 Internal FPGA Architecture . . . 8
8 Experimental Hardware Setup (VC707) . . . 9
9 Block diagram of the FPGA-based intrusion detection system architecture . . . 13
10 Hardware-Software architecture of the FPGA-based intrusion detection system . . . 15
11 Annotated Vivado block diagram . . . 17
12 32-bit word-aligned packet structure showing fixed offsets used by the hardware parser for protocol identification, target extraction, and attack signature detection . . . 18
13 Layout of the `target_stats_t` structure used for per-target traffic tracking . . . 23
14 UART output showing per-window statistics and detected attack types . . . 24
15 Software control flow of the MicroBlaze processor showing packet injection, window boundary detection, and synchronised latching of hardware statistics during real-time IDS operation . . . 25
16 Graphical user interface used to generate network traffic scenarios and attack datasets for IDS evaluation . . . 26
17 Comparison of packet representations. The original PCAP format contains full protocol information, while the compact format stores packets as fixed 32-bit words for FPGA replay. . . . 28
18 A Typical IPv4 TCP header . . . 29
19 Example packets used in testing. Left: normal TCP traffic. Right: TCP SYN packet, showing the SYN flag used for attack detection. . . . 30
20 Extracted destination ports from the SYN packet dataset using `tcpdump`. The repeated destination IP with varying ports indicates SYN-based scanning or probing behaviour. . . . 32
21 Mixed traffic scenario generated using the IDS attack scenario builder . . . 33
22 Window-level output for mixed traffic scenario . . . 34

1 Introduction

Information and Communication Technology (ICT) services are a fundamental part of modern society, supporting critical infrastructure across sectors such as healthcare, government, and finance [1]. As noted by the World Economic Forum, this increasing reliance has expanded the overall potential digital attack surface, making systems more susceptible to malicious disruption [2].

While common cyber threats such as malware, phishing, and ransomware primarily target data integrity and confidentiality, attacks that target system availability remain a major concern. According to the ENISA Threat Landscape 2024 report, Denial of Service (DoS) and Distributed Denial of Service (DDoS) attacks "play a significant role in the cybersecurity threat landscape" [3]. These attacks aim to overwhelm network resources, preventing legitimate users from accessing essential services.

DoS (Denial of Service) attacks are a class of cyberattack in which an attacker attempts to render a system or network resource unavailable to legitimate users by overwhelming it with malicious traffic [4]. A Distributed Denial of Service (DDoS) attack extends this concept by originating from multiple sources simultaneously, significantly increasing the scale and impact of the attack [5]. These attacks are among the most disruptive threats to network availability, capable of causing large-scale service outages and significant financial and reputational damage [6].

In 2020, experts identified 1,560,000 more DDoS attacks than in 2019 as described by [6]. As shown in Figure 1, an attacker can use a network of compromised devices, known as a botnet. Botnets are small malicious programs that are created and controlled by an attacker [7] A botnet consists of multiple infected hosts, each running malicious software that can be remotely controlled by an attacker [7]. This distributed nature provides scalability and anonymity, making DDoS attacks particularly difficult to mitigate using traditional defence mechanisms [8]. DoS attacks exploit vulnerabilities in network infrastructure and can be launched using various methods. The main goal of a DoS attack is to disrupt normal operations, which can cause system downtime which results in service interruptions, revenue loss, and damage to the organization's reputation and cause downtime and potentially inflict financial or reputational damage [9].

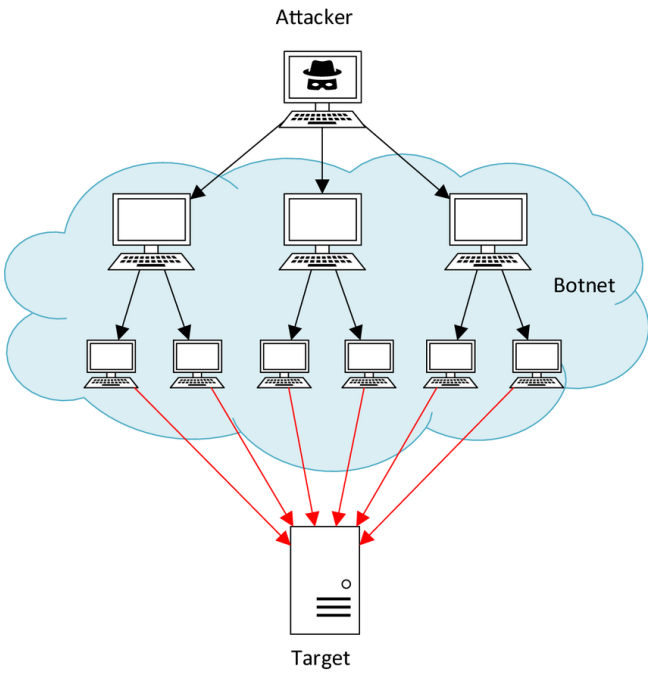


Figure 1: DDoS attack diagram [10].

An effective approach to network security requires consideration of multiple layers within an organisation's infrastructure, as attacks may target different points within the network stack (the hierarchical implementation of networking protocols (e.g., TCP/IP) in software or firmware, managing data transmission from an application down to the physical network). Network attacks are unauthorised actions aimed at compromising the confidentiality, integrity, or availability of data and services. These attacks can broadly be classified as either passive or active. Passive attacks involve the interception of data without modification, whereas active attacks involve altering, disrupting, or destroying network traffic or system resources [11].

DDoS attacks fall into the category of active attacks, as their primary objective is to disrupt normal system operation by overwhelming network resources. Mitigation strategies for such attacks can be implemented using either software-based or hardware-based approaches, each offering different trade-offs in terms of performance, scalability, and cost. Software-based solutions provide flexibility and ease of deployment, while hardware-based approaches offer higher throughput and lower latency, making them better suited to handling high-speed network traffic.

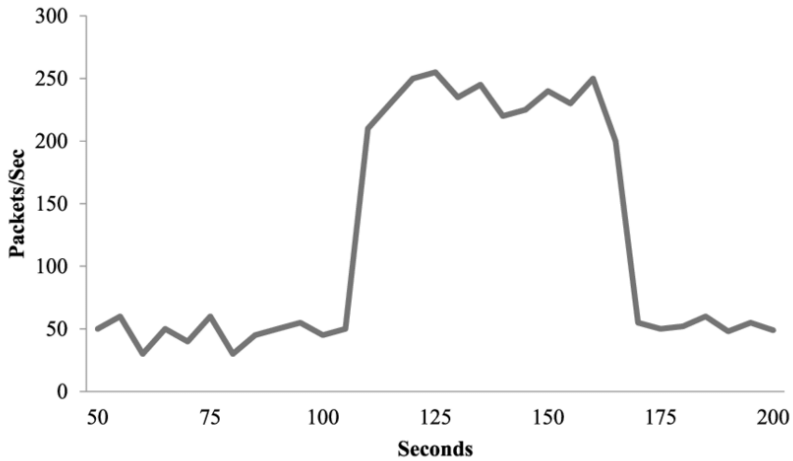


Figure 2: Spike in traffic diagram [12].

Detecting these attacks is a complex task, as it requires monitoring high volumes of traffic, distinguishing between legitimate and malicious requests, and responding to rapidly evolving attack methods. One of the main challenges is the scale of traffic generated during an attack. As shown in Fig. 2, attacks are characterised by sudden and significant spikes in packet rates compared to normal network behaviour. Such surges can overwhelm both network infrastructure and processing systems, making real time analysis difficult.

In addition to high traffic volume, distributed botnets obscure the true origin of attacks and often employ evasion techniques that mimic legitimate traffic patterns, further complicating detection [13]. To address these issues a range of detection/mitigation techniques have been developed. These include anomaly detection, which identifies unusual traffic behaviors that deviate from normal patterns [14], rate limiting, a technique that restricts the number of requests from a single source within a given time frame to prevent overloading a server [15], and traffic filtering, which blocks malicious packets based on predefined rules or signatures [16].

However, implementing these techniques at scale requires significant computational resources. Large-scale DDoS attacks can generate traffic volumes in the terabit per second range, requiring inspection of millions of packets per second (pps). For example, recent reports show that Cloudflare mitigated a 7.3 Tbps DDoS attack, delivering approximately 37.4 terabytes of data in just 45 seconds [17]. Processing traffic at this scale places substantial demands on detection systems, as packets must be analysed in real time. This shows the need for high-performance detection mechanisms capable of operating at line rate, motivating the use of hardware-accelerated solutions such as FPGAs.

Traditional software-based defenses running on general-purpose Central Processing Units (CPUs) face inherent limitations. As described by [18], CPU based systems, even those with multi-core architectures, face challenges in scaling because of the "Von Neumann bottleneck" which is where the CPU must sequentially interrupt its processing to fetch packet data from memory [19]. This can cause the CPU to enter into a state of "receive livelock" where the processor consumes all available interrupts, leaving no resources for other necessary tasks [20]. This can cause the defence system to crash, in fact industry reports show that over 80% of interviewed respondents firewalls have crashed during DDoS attacks [21].

1.1 Problem Statement and Project Solution

The goal of this project was to develop a Field Programmable Gate Array (FPGA) based Intrusion Detection System (IDS) on the Xilinx VC707 platform, capable of detecting Denial of Service (DoS) attacks in real-time. The system implements a packet inspection pipeline designed to overcome the limitations of traditional software-based approaches.

Modern software-based defenses are limited in their ability to process high volumes of network traffic due to

CPU bottlenecks, interrupt overhead, and the sequential nature of packet processing. This motivates the use of FPGA-based architectures, which enable parallel processing and low-latency analysis, making them well-suited for high-throughput network monitoring.

A "monitor-analyse-mitigate" framework is implemented, where incoming packet streams are monitored in real-time, analysed for anomalous behaviour, and used to generate alerts indicating potential DoS attacks. Packet data is replayed from on-chip Block RAM (BRAM) and processed by a hardware packet parser, which extracts key features including protocol type, destination IP address, and TCP flags. These features are then evaluated using a sliding window detection mechanism, where packet counts and protocol-specific activity are aggregated over fixed time intervals.

Detection is performed using configurable thresholds for both volumetric and protocol-based attacks. The system is capable of identifying multiple attack types, including SYN floods, ICMP floods, UDP floods, and TCP-based scan attacks such as NULL and XMAS scans. Upon detection, alert signals are generated in hardware and communicated to an on-chip MicroBlaze processor, which aggregates results and outputs them to a host system via a UART interface.

Testing was conducted using both synthetic and real-world network traffic datasets, including CICIDS2017 and CICIDS2019 captures, to evaluate system behaviour under a range of attack scenarios. The system was assessed in terms of packet classification accuracy, sliding window detection performance, and responsiveness to varying traffic rates.

1.2 Report Structure

The rest of this report is organised as follows:

Section 2 Covers the technical background necessary to understand the proposed solution.

Section 3 Provides a literature review of relevant literature, as well as discussing previous projects trying to achieve similar objectives.

Section 4 Presents the complete system design and development.

Section 5 Presents the testing performed on the system described in Section 4, along with a discussion of the results.

Section 6 Discusses the ethical considerations associated with the project and the developed system.

Section 7 Provoides the conclusion to the report by presenting an evaluation of the project and outlining potential areas for future work.

2 Technical Background

2.1 Layers Of Networks

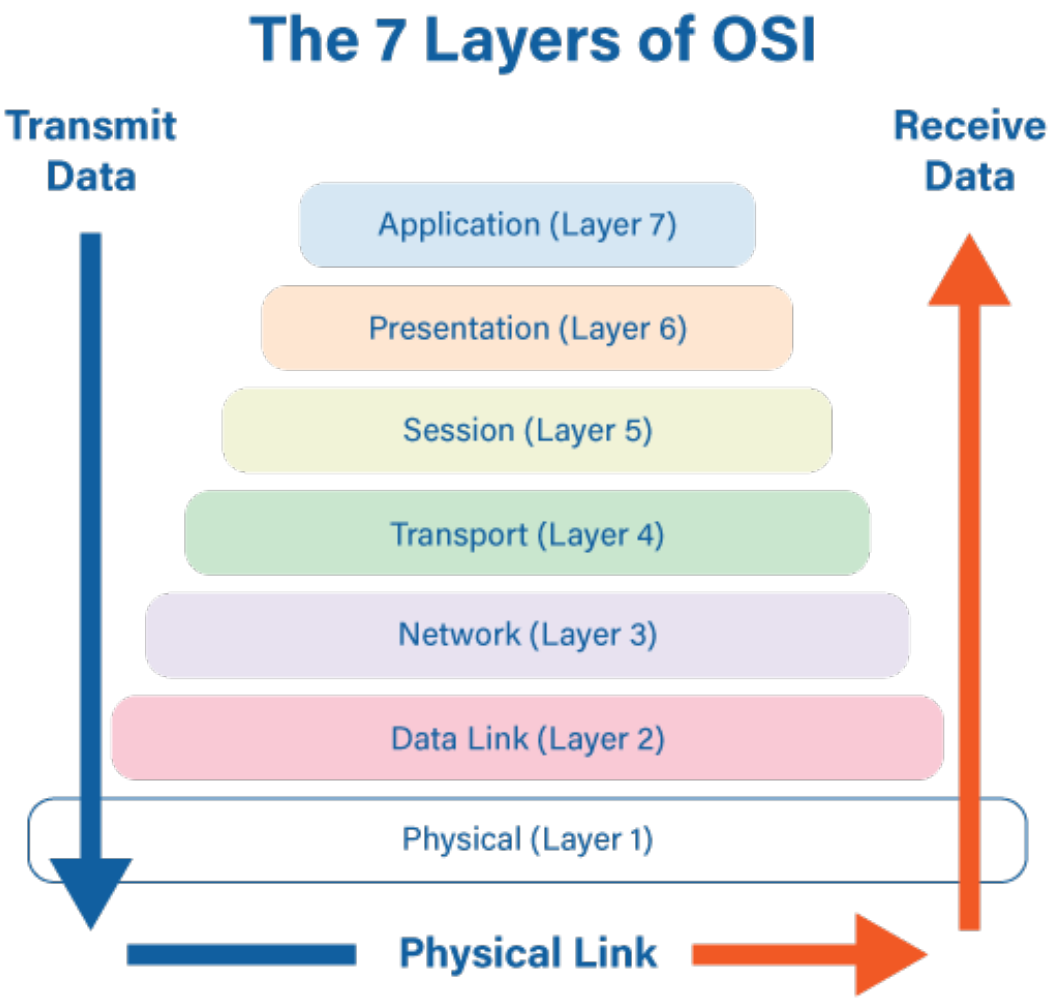


Figure 3: 7 Layers of OSI [22].

In the context of this project, three layers of the OSI model, which is a model of a conceptual framework that divides network communications functions into seven layers, as seen in Figure 3. 3 layers are important, Layers 3 (Network), 4 (Transport), and 7 (Application).

2.1.1 Layer 3

The Network Layer is responsible for the logical addressing and routing of packets between hosts. As shown by Kurose and Ross in Chapter 3 of [23], this layer dictates how data is forwarded through routers to reach its destination.

2.1.2 Layer 4

The Transport Layer ensures end-to-end communication reliability and flow control, mainly through TCP and UDP [24]. Unlike Layer 3, which focuses on volume, Layer 4 DoS attacks target the memory structures used to track active connections.

2.1.3 Layer 7

The Application Layer sits at the top of the OSI model and is responsible for integrating network services with end user software, such as web browsers. As described by Nikhil et al. vulnerabilities at this layer are unique because they target the computational resources of the server rather than the network bandwidth. At this level,

the system must fully process requests, for example, loading a webpage or running a database query. The main danger lies in the asymmetry of resource consumption. A single Hypertext Transfer Protocol (HTTP) request can force the victim server to allocate significant CPU and memory to generate a response, allowing attackers to exhaust server capacity without requiring a high volume of network traffic [25].

2.2 Types Of DoS Attacks

DoS attacks come in different types, each targeting specific weaknesses in the network. To build an effective detection system, it's important to review how these attacks work. Typically, these attacks are divided into three main types: Volumetric attacks, Protocol exploits, and Application-layer attacks. The next sections explain how each type works and how they impact systems.

2.2.1 Application Layer Attacks

While Volumetric and protocol attacks focus on disrupting the network infrastructure (Layers 3 and 4), Application Layer attacks focus on the highest level of the OSI model (Layer 7). Unlike the volumetric/protocol attacks, which try to overwhelm network bandwidth, these attacks target a web application's services, making them unavailable to users, which results in revenue loss for an organisation. Just one minute of downtime can cost a single company as much as $\$22,000$ in lost revenue. [26].

Because these attacks consume server resources rather than network capacity, they can be effective with a much lower volume of traffic, making them significantly harder to detect than volumetric floods [27].

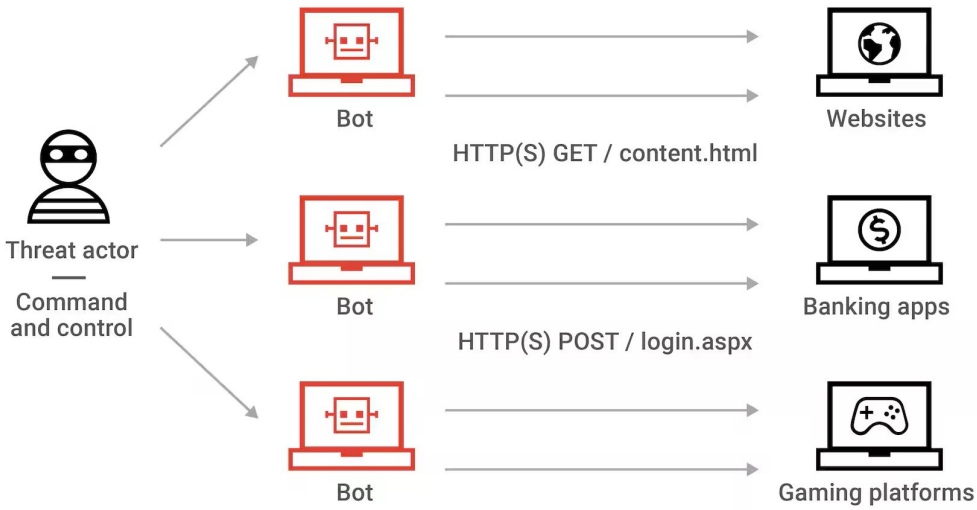


Figure 4: Botnet executing HTTP GET and POST floods against application targets [28].

2.2.1.1 HTTP Floods

As shown in Figure 4, the threat actor uses a command and control server to direct bots to interact with specific parts of a web application. The traffic often mimics legitimate user behavior, such as a web browser requesting a page or a banking app submitting a form.

1. **HTTP GET Flood:** As shown in the top pathway, bots flood the target with requests for specific pages (e.g., `content.html`). While these requests are cheap for the bot to send, they force the server to exhaust its CPU by retrieving files and executing scripts for every single request, effectively overwhelming the system [29].
2. **HTTP POST Flood:** In the lower pathways, bots target sensitive areas like banking apps or login portals (e.g., `login.aspx`). This is an "asymmetric" attack: it requires little effort from the attacker but forces the victim server to perform expensive tasks—such as parsing data, querying databases, and verifying credentials—which drains resources significantly faster than a GET flood [30].

To a standard firewall, a HTTP GET request from a bot looks identical to one from a legitimate user, making it challenging to distinguish between the two without looking into it further. [30] [31].

2.2.2 Protocol Exploits

Protocol exploits target weaknesses in the specific design of network protocols (Layer 3 and Layer 4) rather than simply overwhelming the network pipe with raw volume as seen in the volumetric attack section above. By exploiting the communication protocols defined in TCP/IP, attackers can deplete essential server resources like memory buffers, CPU processing power, and state tables.

2.2.2.1 TCP SYN Flood One of the most common forms of DoS attacks is the TCP SYN Flood. This attack targets the stateful nature of the TCP "three way handshake". A three way handshake is where the client sends a SYN request. The server responds with a SYN ACK, (Synchronise Acknowledge), which is a response sent by the server to the SYN, acknowledging the request and signaling readiness to establish the connection. Finally the client completes the handshake with an ACK (Acknowledge), which is a request sent by the client to confirm the connection is established, completing the three way handshake.

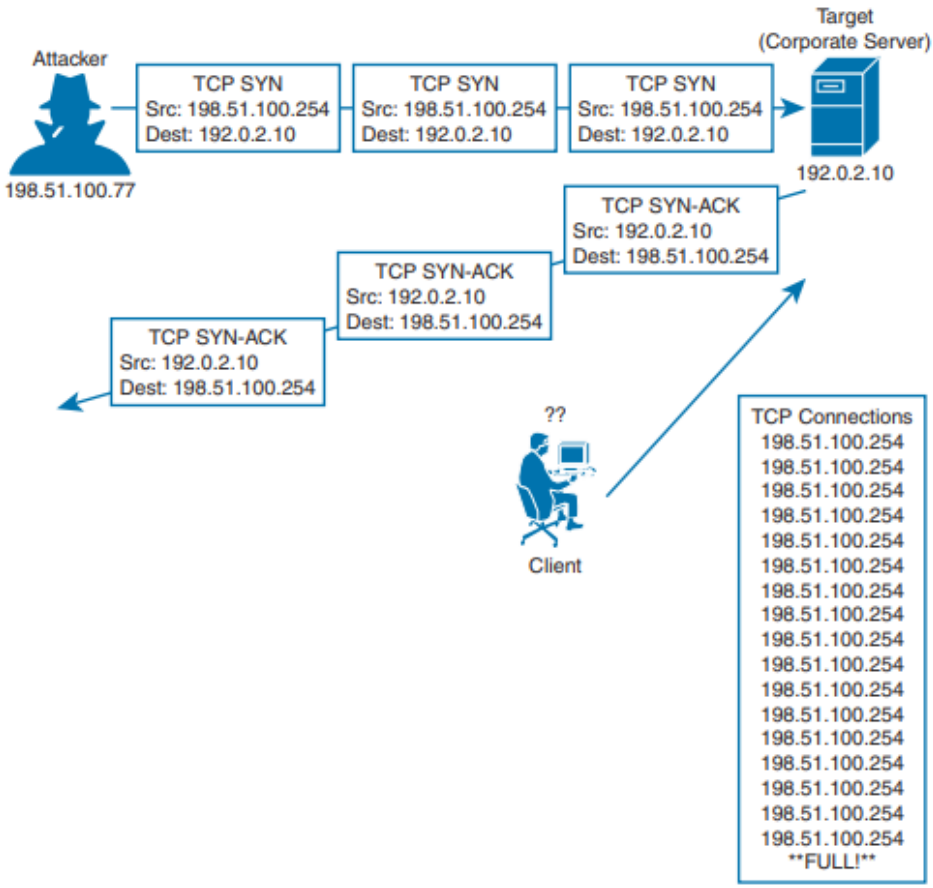


Figure 5: Diagram of a SYN Flood where the attacker uses a spoofed IP to fill the target's connection table [32].

As shown in Figure 5, the attacker sends a rapid succession of SYN packets to the target. The diagram also shows how the attacker pretends to be a trusted IP address on the network (IP Spoofing) rather than using their real IP, which is something that could be looked into for future work.

This method has an asymmetric nature, it requires a small amount of computational power for the attacker to send these fake requests, but it forces the victim server to use a lot of memory resources to track them [33]. In a normal handshake, the server replies with a SYN ACK and allocates memory in a structure called the Transmission Control Block (TCB) to store the connection state. It then waits for a final ACK from the client [33].

In a SYN flood with IP spoofing, because the attacker is pretending to be someone else, the server sends the response into the void and never receives the final ACK. This leaves the server with a "half open" connection. Since the server's backlog queue (the buffer for pending connections) has a finite size, the flood quickly fills all available slots. Once the backlog is full, as seen in the right of the diagram, the server drops all incoming requests, rendering it inaccessible to the legitimate user [34].

2.2.3 Volumetric Attacks

Volumetric attacks, often measured in bits per second (bps), are the most common form of DoS. Unlike application layer attacks that target server logic, the main objective of a volumetric attack is for an attacker to disrupt user access to a service by sending traffic that reduces the available capacity for legitimate users [35]. By generating massive congestion, legitimate traffic is prevented from using the network link, resulting in a denial of service.

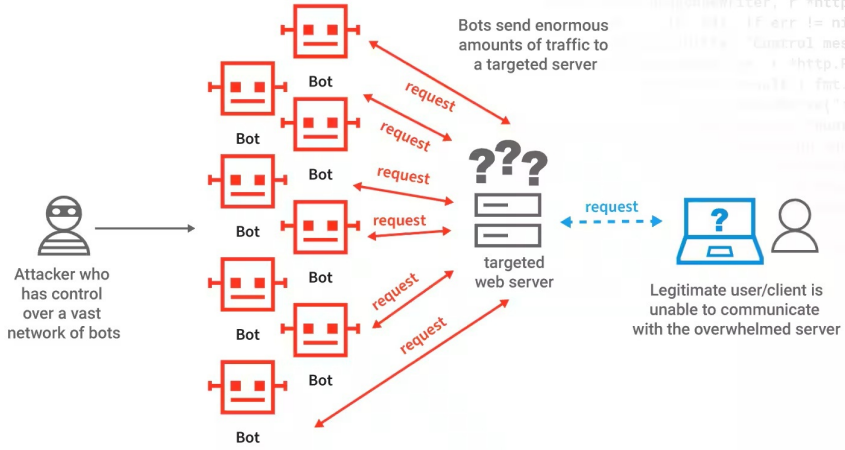


Figure 6: Architecture of a Volumetric DDoS Attack using a Botnet [36].

As shown in Figure 6, while the diagram shows a botnet based DDoS attack, a similar volumetric attack can also be launched from a single machine, which is the case in DoS attacks. In these attacks, it's not the content of the request that matters, but the sheer volume of traffic. The target server's network interface or the upstream router becomes overwhelmed, preventing the legitimate users, as shown in the diagram, from establishing a connection.

At a low level, volumetric attacks typically operate on Layer 3 (Network) and Layer 4 (Transport) of the OSI model. Common attack methods include UDP floods, which is where random ports on a server are flooded with UDP packets, making the server repeatedly search for and respond to non existent applications at those ports. ICMP floods are also common, ICMP is Internet Control Message Protocol, a network layer diagnostic protocol used to report errors and check connectivity (like "Ping"), differing from TCP/UDP as it is used for network management rather than data transport and does not use port numbers. ICMP floods are where the server is flooded with ICMP requests from multiple fake IP addresses. As the server processes and responds to these bogus requests, it becomes overloaded and unable to handle legitimate requests [37]. To increase the volume of traffic, attackers often use Reflection and Amplification techniques.

2.2.3.1 Amplification In an amplification attack, the attacker sends a small request to a third party server (such as a DNS resolver (a server or service that helps translate domain names) or NTP server (A server that provides accurate time information to other devices on a network.)) but spoofs the source IP address to match the victim's IP. The third party server then replies to the victim with a response payload that is significantly larger than the initial request. This allows an attacker with limited bandwidth to generate a massive volumetric flood against the target [38], [39].

2.3 FPGA Architecture

Field-Programmable Gate Arrays (FPGAs) are devices that can be electrically programmed in the field to become almost any kind of digital circuit or system **farooq˙fpga˙2012**. They consist of a matrix of Configurable Logic Blocks (CLBs) connected together via a programmable routing fabric, which can be wired to perform complex digital functions, as shown in Figure 7. Unlike a CPU, which executes instructions sequentially, the internal circuitry of an FPGA executes logic in parallel. The hardware reconfigurability of an FPGA is what distinguishes it from a standard processor, offering a balance between the flexibility of general purpose processors and the performance of Application Specific Integrated Circuits (ASICs) [40].

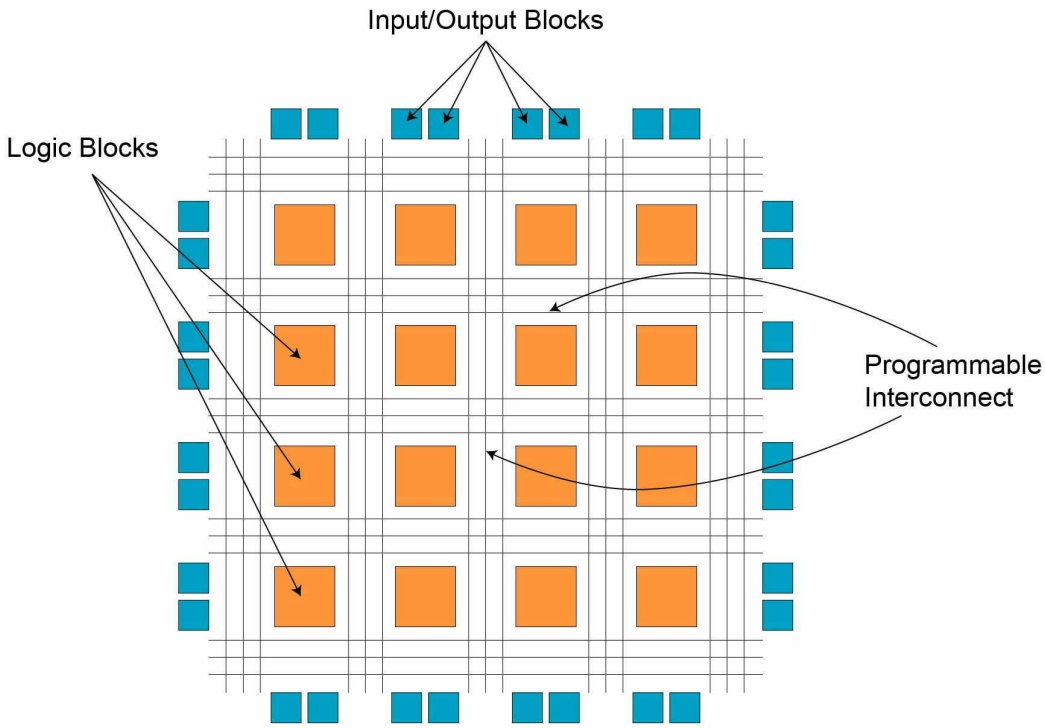


Figure 7: Diagram showing the internal architecture of an FPGA, showing CLBs connected via programmable interconnects **admin˙fpga˙2023**.

FPGAs consist of several fundamental components [41]:

Programmable Logic: The core computational fabric of the FPGA, traditionally composed of Configurable Logic Blocks and Look-Up Tables (LUTs). This layer implements the custom digital logic and sequential operations as defined by the user [41].

Programmable Routing: This is a network of wires and programmable switches (interconnects) that links logic blocks together. It acts as the device's "nervous system," routing signals between the programmable logic, DSPs, and I/O (Input Output) pins [41].

Programmable IO: The input/output blocks (IOBs) that interface the internal logic with physical device pins. These are configurable to support various voltage standards and protocols (e.g DDR) for communicating with external hardware [41].

DSP Blocks: Specialised arithmetic units embedded in the fabric. Unlike general logic, these are optimised to perform high-speed mathematical operations (like multiply accumulate) efficiently, which is critical for signal processing tasks [41].

System-Level Interconnect (Network-on-Chip): The communication infrastructure used to manage data flow between subsystems [41]. This can be implemented in two ways:

Hard NoC: Dedicated, fixed-silicon routers embedded in the chip. As detailed by Abdelfattah and Betz **s˙networks˙chip˙2014**, these hardened structures are up to 23× more area-efficient and 6× faster than programmable logic solutions.

Soft NoC: Interconnects constructed using the FPGAs general-purpose logic (LUTs) and routing fabric (e.g. the AXI Interconnect linking the MicroBlaze soft-processor to the detection logic).

While Hard NoCs offer superior performance, the Virtex-7 architecture used in the VC707 is reliant on Soft NoCs. As a result, a portion of the programmable logic must be allocated to implementing these data buses to ensure non blocking communication between the Packet Monitor and the MicroBlaze controller..

Interposers: A silicon integration layer found in 2.5D ICs (A type of semiconductor packaging technology that allows multiple chips to be integrated in a single package with a vertical stacking arrangement). It allows multiple memory chips (like HBM (High Bandwidth Memory)) to be mounted side-by-side in a single package, which allows massive logic scaling and higher bandwidth [41].

2.3.1 Xilinx Overview

This project uses the Xilinx 7-Series FPGA architecture, specifically the Virtex-7 family. Unlike the Zynq architecture which integrates a hard core ARM processor, the Virtex-7 is a pure FPGA fabric optimised for massive logic capacity [42].

Because this architecture lacks a dedicated Processing System (PS), any necessary software tasks (such as logging or high-level policy management) are implemented using a MicroBlaze soft-processor instantiated within the programmable logic. Development is carried out using the Xilinx Vivado Design Suite, which serves as the main environment for synthesising Hardware Description Languages (HDL) into a bitstream that configures the physical logic gates [43].

2.3.2 VC707 Platform

The physical implementation of the project is deployed on the Xilinx VC707 Evaluation Kit, as shown in Figure 8. This high performance platform is built around the Virtex-7 XC7VX485T FPGA, a device optimised for high bandwidth connectivity and logic intensive applications. Unlike the Zynq-7000 series which integrates a hard wired ARM processor and PL [44], the VC707 relies entirely on reconfigurable logic [45]. This architecture was selected because it eliminates Operating System (OS) overhead, allowing for line-rate packet processing which is important when detecting attacks that occur in less than a second.

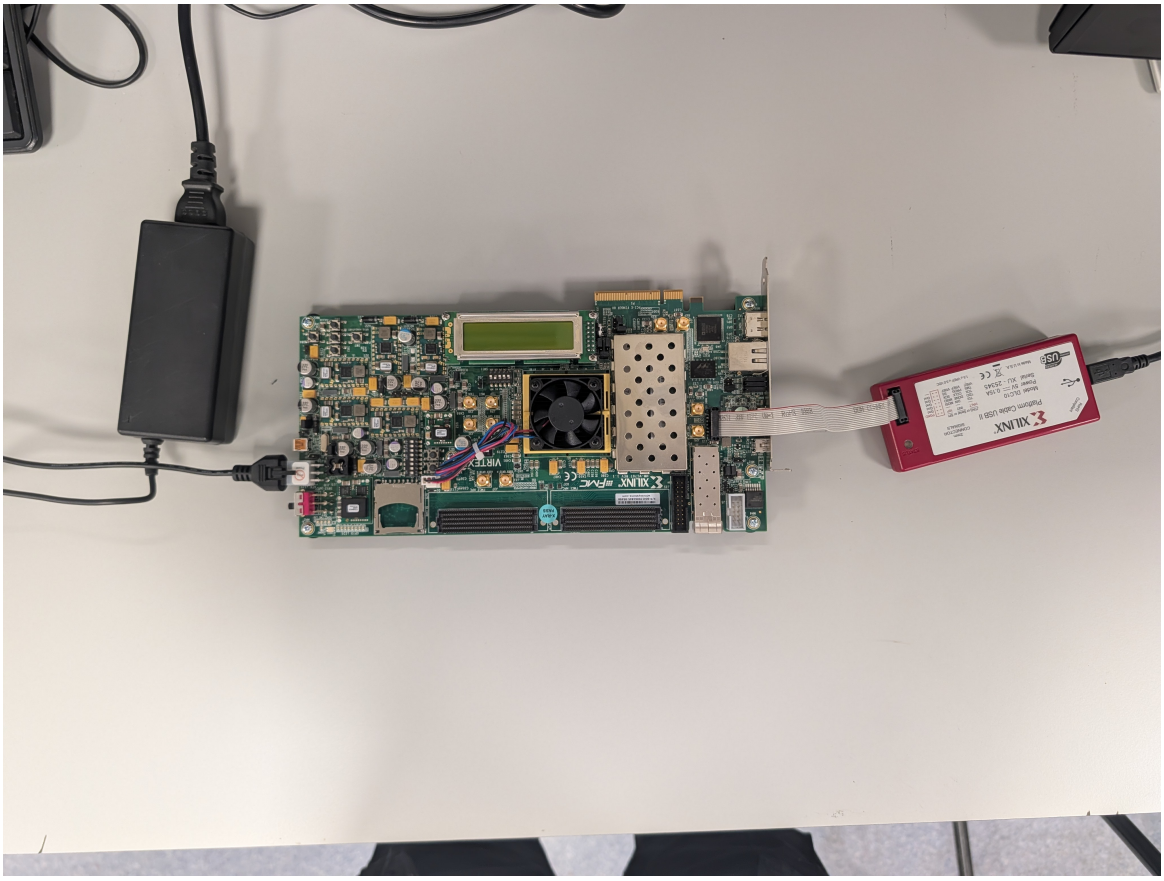


Figure 8: Experimental hardware setup featuring the Xilinx VC707 Evaluation Kit. Left: The 12V power supply. Right: The red Xilinx Platform Cable USB II (DLC10) programmer connected to the JTAG header for bitstream config and debugging.

Key features of the platform used in the project include:

Virtex-7 FPGA Fabric: The central device (visible under the cooling fan in Figure 8) provides 485,760 logic cells and 2,800 DSP slices [42]. This high logic density is important for implementing parallel detection algorithms, for example simultaneous tracking of multiple network flows.

SFP+ Networking Interfaces: The board includes Quad SFP+ (Small Form factor Pluggable) cages (visible on the right side of the board). These interfaces allow the system to directly receive network traffic from optical or copper, bypassing the latency typically introduced by standard software stacks(a collection of

software components or technologies that work together to provide a set of functionalities for a specific application). This makes sure that the detection logic processes data at full "wire-speed" (up to 10Gbps for SFP+, up to 1Gbps for SFP).

DDR3 SODIMM Memory: The board includes 1GB of DDR3 (Double Data Rate) memory connected directly to the FPGA logic. In this design, it is used as a high speed circular buffer to temporarily store packet headers while the detection logic calculates the flow statistics.

JTAG Configuration Interface: The system is programmed using the Xilinx Platform Cable USB II (DLC10), visible on the right side of Figure 8 (the red device). This interface enables bitstream uploading via Vivado and real-time hardware debugging.

User I/O (Alert System): The board features an LCD character display and status LEDs (visible near the top-left). In this project, these can serve as the physical output for the Detection System, providing visual confirmation when a specific threshold is breached and a DoS attack is detected.

One of the main advantages of using FPGA architecture in network security is their ability to run multiple operations at the same time. Unlike general-purpose processors that reuse a shared Arithmetic Logic Unit (ALU) to execute instructions sequentially, an FPGA can instantiate multiple independent logic circuits that operate concurrently. This allows the creation of pipelined data paths, where each stage of packet inspection (e.g. parsing, feature extraction, and counting) is mapped to dedicated hardware.

In the context of this project, the hardware design enables line-rate processing in principle. Packet data can be analysed as it streams through the device, with header fields extracted and multiple detection counters updated within a single clock cycle.

Because these operations are implemented as fixed RTL structures rather than software routines, the system exhibits deterministic latency, independent of traffic volume. However, in the current implementation, packet replay is performed from BRAM via the MicroBlaze processor, which introduces a software-controlled bottleneck and limits the achievable throughput.

This is particularly important for Denial of Service detection, where sudden traffic surges can overwhelm software-based systems. By eliminating shared resource contention and interrupt overhead, the FPGA-based approach avoids packet loss and ensures consistent inspection coverage even under high load.

2.4 MicroBlaze Processor

The MicroBlaze processor is a 32 bit RISC (Reduced Instruction Set Computing, a CPU architecture that uses a small, highly optimised set of instructions) soft core processor developed by AMD, designed to be instantiated within the programmable logic fabric of an FPGA. Unlike hard-core processors such as the ARM cores in the Zynq family, the MicroBlaze is synthesised from IP blocks, which allows it to be tailored to specific requirements. This flexibility makes it suitable for embedded control tasks, packet inspection, and integration with custom hardware accelerators [46], [47]. Architecturally, MicroBlaze follows a Harvard style, meaning that the program memory and data memory are separate, with separate instruction and data buses. It can be also configured with instruction and data caches, hardware multiplier and divider units, and a floating point unit if needed. Communication with memory and peripherals is achieved through Local Memory Bus (LMB) interfaces. This modularity makes sure that the processor can be optimised for performance.

In the context of this project, the MicroBlaze processor will provide a lightweight supervisory layer that works alongside the custom detection logic implemented in the hardware. While the FPGA fabric handles high-speed packet inspection and DoS attack detection, the MicroBlaze can be used to manage configuration registers, log detection events, and provide an interface for debugging and evaluation. This ensures that the detection pipeline remains optimised for speed, while the processor supports higher-level control and monitoring functions. We will go into further detail in Section **??**, where the integration of MicroBlaze into the overall system architecture is described. The setup process begins in the Vivado Design Suite, where the MicroBlaze core is instantiated using the IP Integrator. The configuration wizard allows the designer to select the implementation type and enable the necessary features. Once the processor is defined, peripherals such as memory controllers, UART interfaces, GPIO modules, and interrupt controllers are added to the design and connected via AXI interconnects. After synthesising the hardware description is then exported to the Vitis IDE. Within Vitis, application projects are created, and C/C++ programs are developed. These programs are then compiled and loaded onto the FPGA via JTAG, which you can see in Figure 8. Debugging is supported directly within Vitis, allowing the designer to monitor memory usage, set breakpoints, and evaluate how efficiently the program is running/executed [48]. Compared to hard-core processors such as ARM Cortex-A9 or Cortex-A53, the MicroBlaze offers greater configurability but a lower raw output of performance. In this project, its role is

to provide a flexible and programmable control environment that enhances the usability and evaluation of the intrusion detection system.

2.5 AXI Interconnects

AXI4-Stream is a high-speed communication protocol designed for transferring data as a continuous stream between hardware modules. Unlike memory-mapped interfaces, it does not use addresses, making it well suited to applications where data flows sequentially, such as packet processing.

AXI4-Streams form the data path of the system, carrying raw packet data through the detection pipeline. Because it does not rely on address-based transactions, it minimises protocol overhead and allows continuous high-throughput data transfer.

The interface operates using a VALID/READY handshake mechanism. In this project, the `TVALID` signal indicates that a 32-bit word is present on the bus, while `TLAST` marks the end of a packet. This allows the hardware parser to maintain packet boundaries and reset its internal state accordingly.

The absence of a hardware backpressure signal in the GPIO-based implementation necessitates software-level timing control, which is achieved using short synchronisation delays during packet injection.

2.6 Hardware-Software Co-Design

A key design principle of this project is dividing functionality between the data plane and the control plane, forming a hardware-software co-design.

The Data Plane, implemented in FPGA logic, is responsible for time-sensitive operations such as packet parsing, protocol classification, and real-time counter updates. These tasks are executed at the system clock frequency, allowing consistent throughput.

The Control Plane, implemented on the MicroBlaze processor, manages system configuration, dataset replay, and result reporting. These tasks involve more complex control logic but are not latency-sensitive, making them well suited to software implementation.

This partitioning allows the system to exploit the strengths of both domains, the parallel processing capability of hardware and the flexibility of software. As a result, the IDS achieves high-performance packet inspection while maintaining adaptability for testing and evaluation.

2.7 Scapy and Traffic Generation

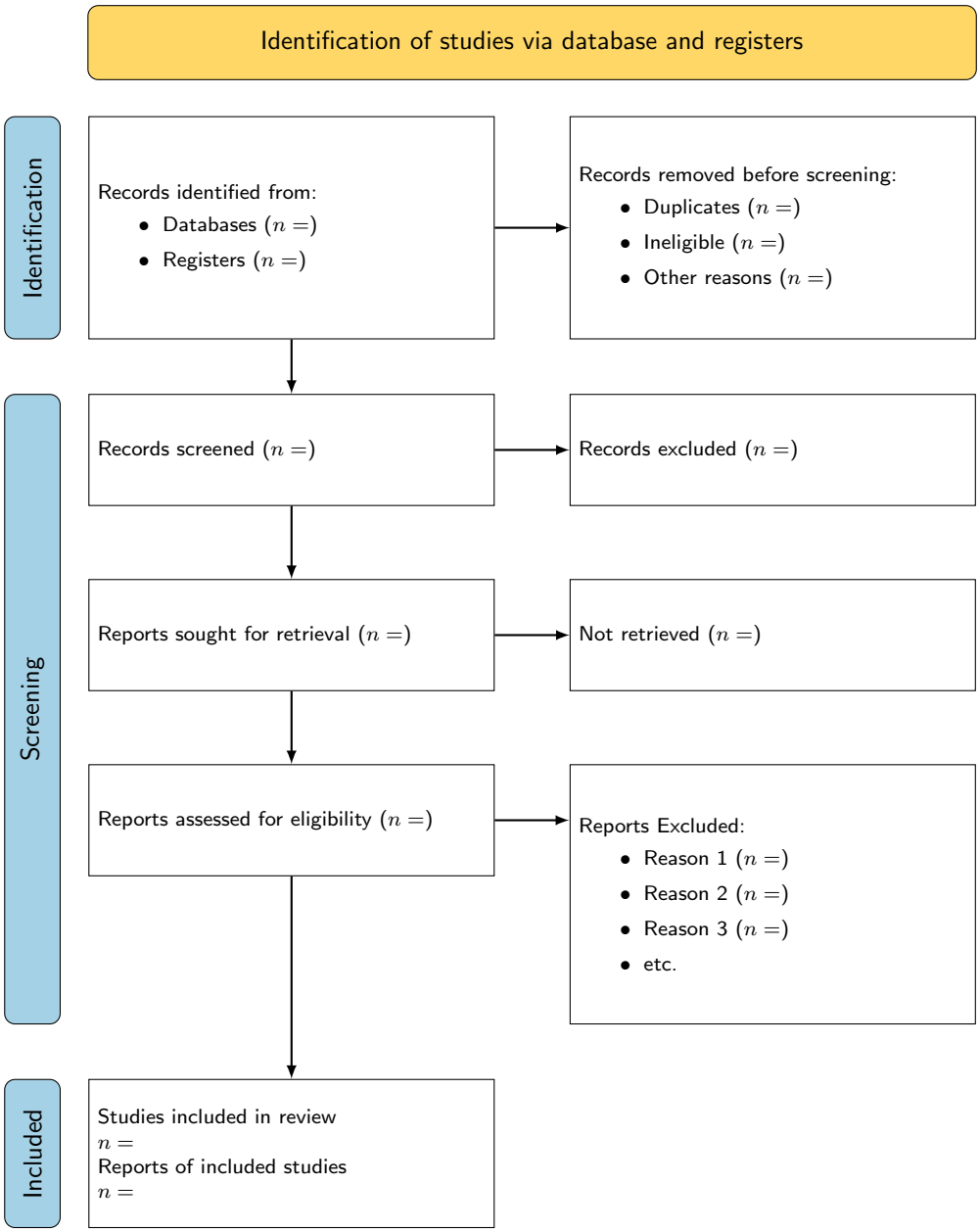
Scapy is a Python based packet manipulation tool. It allows users to create, send, capture, and analyse network packets. Unlike traditional traffic generation tools, Scapy can provide access to packet headers, which allows the creation of custom traffic patterns for testing and research purposes [49].

The framework supports a wide range of protocols, including Ethernet, IP, TCP, User Datagram Protocol (UDP), and Internet Control Message Protocol (ICMP). By combining packet crafting with Python scripting, Scapy can be used to simulate testing scenarios such as flooding attacks.

In the context of this project, Scapy will used to generate synthetic traffic that simulates Denial of Service attacks. For example, a TCP SYN flood can be scripted to repeatedly send packets with the SYN flag set. By running these scripts within a Jupyter Notebook, Scapy can integrate with the FPGA detection framework, allowing control over attack parameters and timestamps.

3 Literature Review

3.1 Prisma



$packetparsing- > https://www.xilinx.com/publications/about/ANCS_final.pdf$

4 System Design and Development

This section describes the design and development of the FPGA-based intrusion detection system, including the key architectural decisions, hardware implementation, and software integration required for real-time DoS detection.

The system is designed as a packet processing pipeline, with stages for packet replay, parsing, feature extraction, and sliding window-based detection.

For better clarity, this section is organized into subsections, each corresponding to a main component of the system.

- High-Level System Overview (4.1)
- Packet Replay and Data Source (4.2)
- IDS Module (4.3)
- MicroBlaze (4.5)
- GUI (4.6)

A high-level overview is presented first, providing context for the descriptions that follow.

4.1 High Level System Overview

The FPGA-based intrusion detection system is designed as a data processing pipeline for real-time analysis of network traffic. Rather than relying on sequential software processing, the system performs packet inspection directly in hardware, enabling high-throughput and low-latency detection of DoS attacks.

4.1.1 System Architecture

A block diagram of the overall system architecture is shown in Fig. 9.

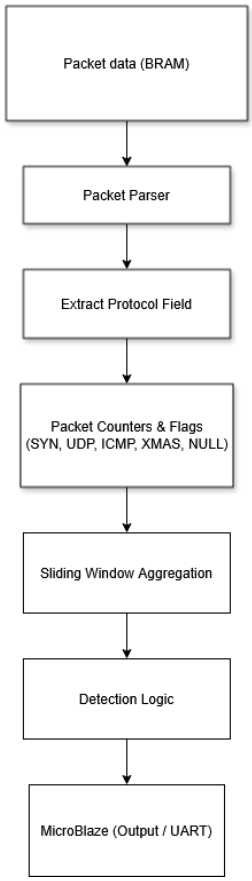


Figure 9: Block diagram of the FPGA-based intrusion detection system architecture

To support communication between the MicroBlaze processor and the IDS hardware module, a set of standardised interfaces is used. These signals define how packet data, configuration parameters, and detection

results are exchanged between the software and hardware components. The main signals used in the system are summarised in Table 2.

Table 1: Signal interface between MicroBlaze and IDS hardware

Signal Name	Interface	Direction	Description
`tdata[31:0]`	AXI-Stream	MB → HW	32-bit packet data word transmitted from the MicroBlaze to the IDS module.
`tvalid`	AXI-Stream	MB → HW	Indicates that the current data word is valid and should be processed.
`tlast`	AXI-Stream	MB → HW	Marks the final word of a packet, allowing the parser to reset for the next packet.
`thresholds[31:0]`	AXI-Lite	MB → HW	Contains detection thresholds for flood and volumetric analysis, configurable at runtime.
`status_out[31:0]`	AXI-Lite	HW → MB	Status register containing detection flags and aggregated traffic statistics.
`dst_ip_out[31:0]`	AXI-Lite	HW → MB	Extracted destination IP address from the most recent packet.

4.1.2 Data Flow Through the System

Network traffic is supplied to the system by the MicroBlaze processor, which replays pre-processed packet datasets stored as C header files into the FPGA packet input interface (as can be seen in listing 1). As packets are streamed into the system, a hardware packet parser extracts key fields such as protocol type, destination IP address, and TCP flags. These features are then passed to a detection module, which implements a sliding window-based approach to analyse traffic patterns over fixed time intervals.

Within each detection window, packet counts and protocol-specific activity are accumulated in hardware and compared against predefined thresholds to detect potential attacks. When a threshold is exceeded, the hardware raises the corresponding detection flags and makes them available through status registers.

These status signals are then read by the MicroBlaze processor, which identifies the target and formats the results for reporting.

The output from the hardware detection logic is communicated to a MicroBlaze processor, which performs higher-level aggregation and reporting. Detected attacks and traffic statistics are then transmitted to the serial monitor via a UART interface.

4.1.3 Hardware-Software Partitioning

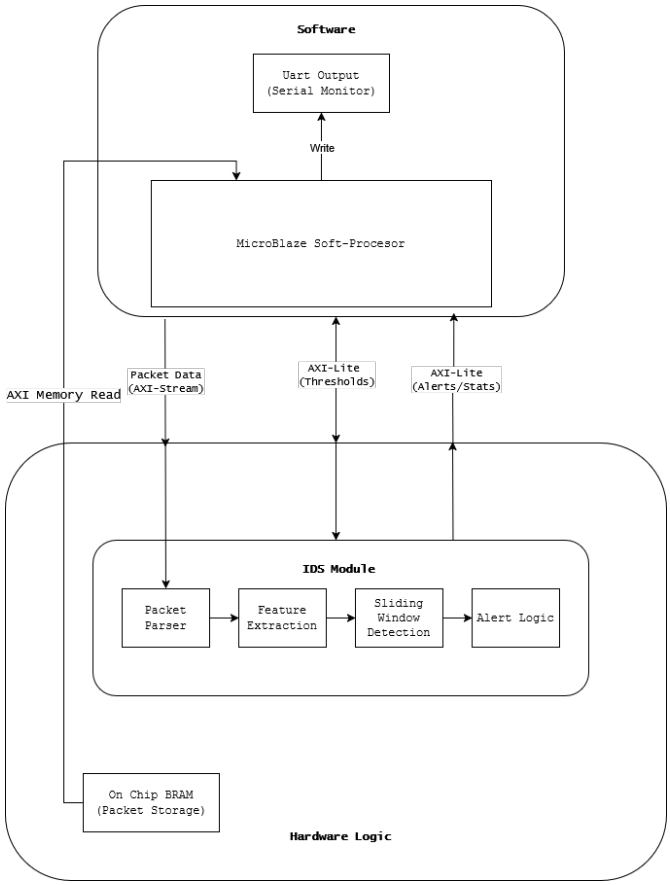


Figure 10: Hardware-Software architecture of the FPGA-based intrusion detection system

As shown in Fig. 10, the system combines both hardware and software components.

The software layer, implemented on a MicroBlaze processor, is responsible for managing packet datasets, transmitting packet data to the hardware, and reporting detection results via UART.

The hardware layer is made up of of a module implemented in Vivado and then is programmed onto the FPGA. This module handles packet parsing, feature extraction, and detection as data is processed in a continuous stream. By offloading these operations to hardware, the system is able to process packets with low latency and high throughput.

4.1.4 Communication Interfaces and Interconnects

To facilitate the hardware-software design, a standardised communication interface was established between the MicroBlaze processor and the IDS hardware module. This interface allows high-speed packet injection while enabling runtime configuration and monitoring.

Table 2: Signal interface between MicroBlaze and IDS hardware

Signal Name	Interface	Direction	Description
`tdata[31:0]`	AXI GPIO	MB → HW	Raw 32-bit packet word from dataset.
`tvalid`	AXI GPIO	MB → HW	Indicates valid packet data.
`tlast`	AXI GPIO	MB → HW	Marks end of packet; resets parser.
`thresholds[31:0]`	AXI GPIO	MB → HW	Detection thresholds for flood and volumetric logic.
`status_out[31:0]`	AXI GPIO	HW → MB	Detection flags and window statistics.
`dst_ip_out[31:0]`	AXI GPIO	HW → MB	Extracted destination IP for reporting.

4.1.5 Platform Integration and Physical Implementation

Communication between the software and hardware components is achieved using AXI-based interfaces.

An AXI-Stream interface is used to transmit packet data from the MicroBlaze processor to the IDS hardware module. This allows packets to be processed as a continuous stream without requiring full buffering.

AXI-Lite interfaces are used for configuration and status monitoring. The MicroBlaze writes detection thresholds to hardware registers and reads back detection results and traffic statistics via memory-mapped interfaces. The main subsystems, shown in Fig. 11 are as follows:

(1) Clock and Reset System (orange): Provides a stable clock signal and synchronised reset for all components, ensuring reliable system operation.

(2) MicroBlaze Processor System (green): Acts as a software-based MicroBlaze processor, serving as the central control unit for packet replay, configuration of detection thresholds, and collection of detection results.

(3) AXI Interconnect (brown): Allows communication between the MicroBlaze processor and all peripheral modules through AXI interfaces.

(4A) AXI GPIO Inputs (red): Used to transmit packet data, control signals, and configuration parameters from the processor to the hardware modules. In particular, **(4A1)** corresponds to control and configuration interfaces (e.g. thresholds and trigger signals), while **(4A2)** is used for packet data input, where individual packet words are streamed from the MicroBlaze processor into the IDS hardware.

(4B) AXI GPIO Outputs (purple): Return detection results and traffic statistics from the hardware modules back to the processor.

(5) IDS Hardware Module (blue): Implements packet parsing, feature extraction, and attack detection in hardware, processing incoming data streams in real time.

(6) UART Output Interface (black): Sends detection results to an external system via UART.

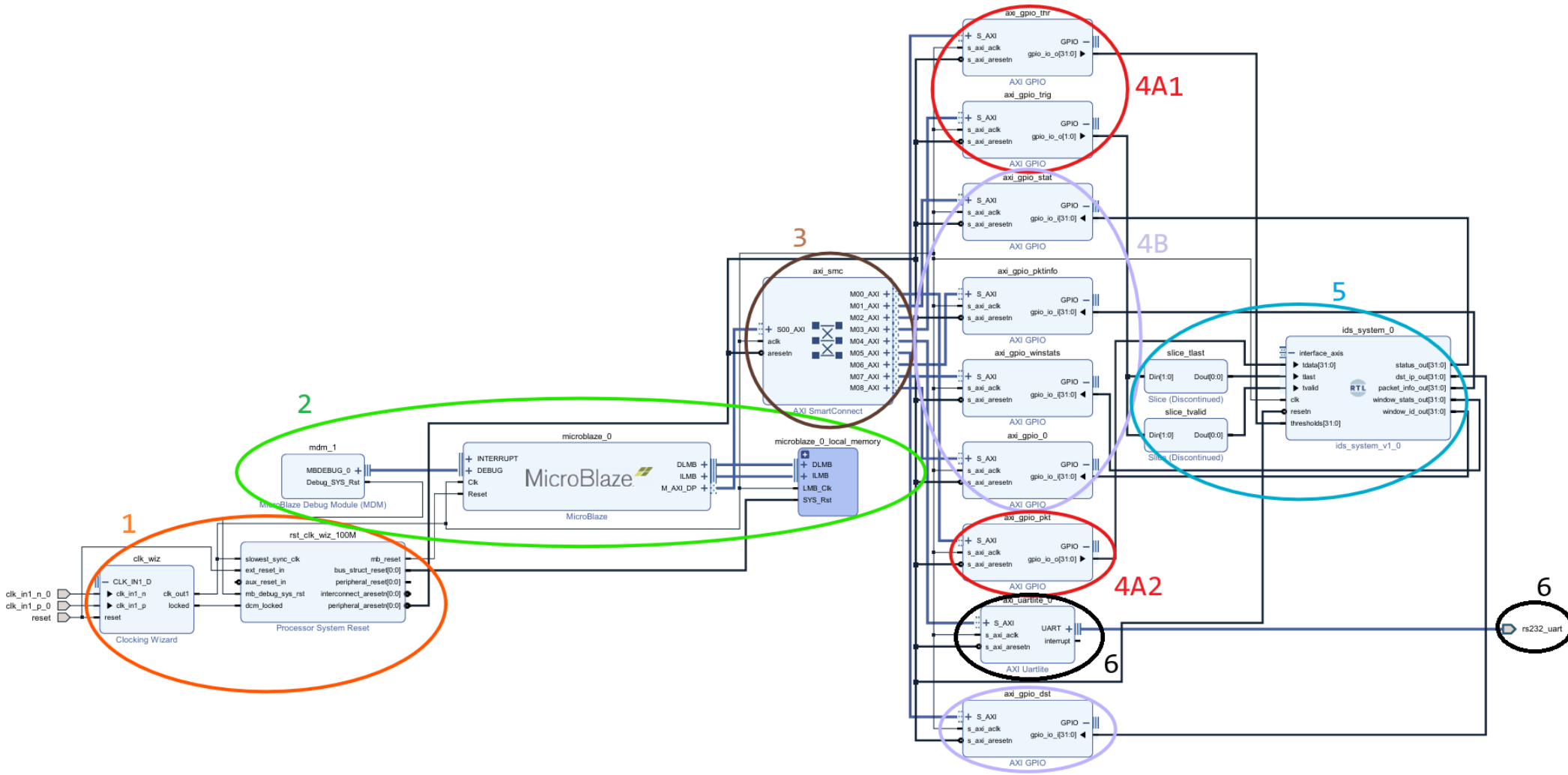


Figure 11: Annotated Vivado block diagram

4.2 Packet Replay and Data Source

The IDS uses on-chip Block RAM (BRAM) as a source of network traffic for testing. Packet data is pre-defined and stored in memory, then replayed into the FPGA to simulate real network traffic conditions.

Each packet is stored in a C header file as a sequence of 32-bit words, the format of which can be seen in Listing 1. The active dataset is selected in the Vitis application, allowing different packet sets to be tested by changing the included header file and associated dataset definition. During execution, the MicroBlaze processor reads the selected packet records from software memory and writes each 32-bit word to the FPGA via memory-mapped AXI GPIO interfaces.

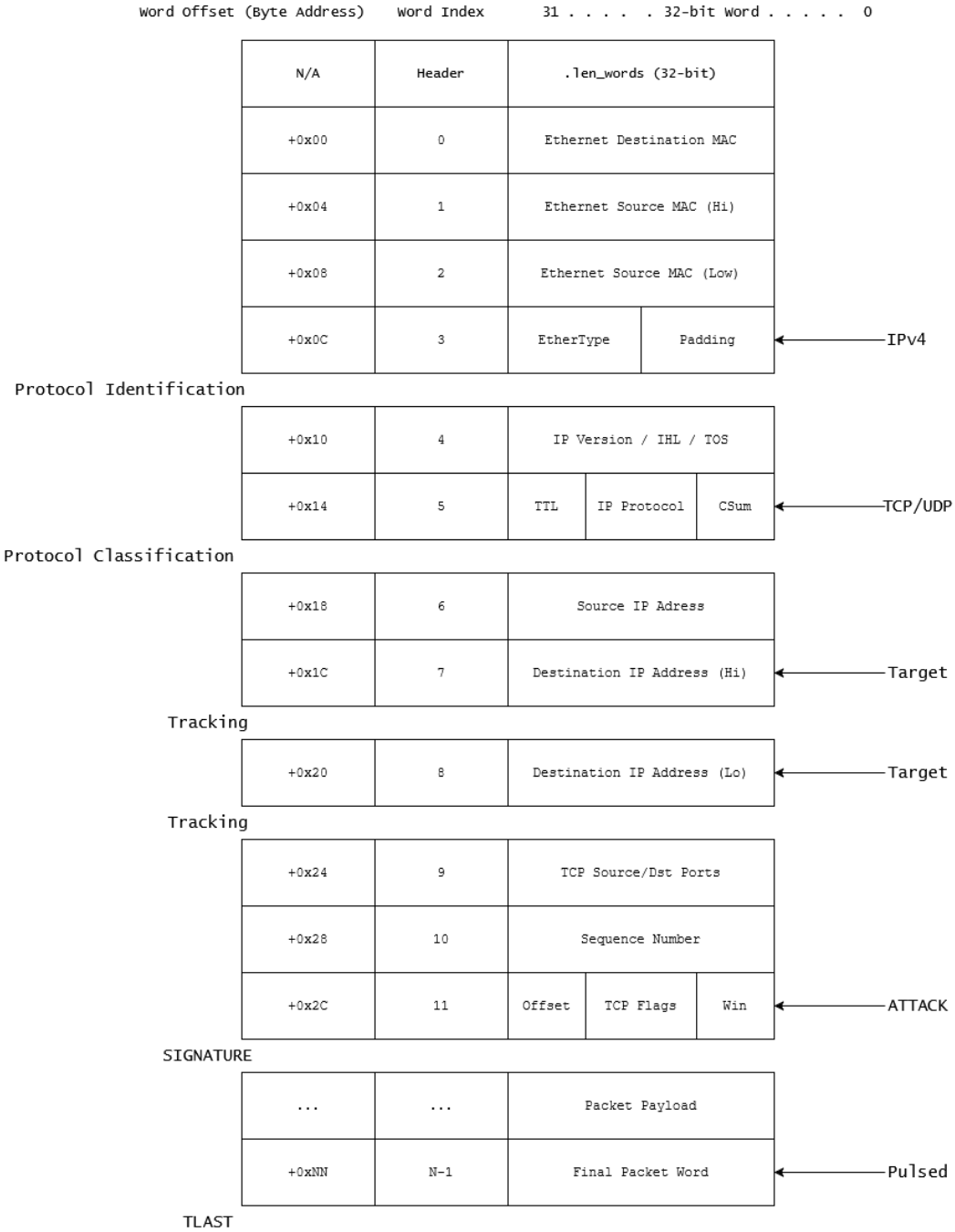


Figure 12: 32-bit word-aligned packet structure showing fixed offsets used by the hardware parser for protocol identification, target extraction, and attack signature detection

Packet data is represented as a sequence of 32-bit words, arranged in a fixed layout that matches the expectations of the hardware parser. As shown in Fig. 12, each packet is structured such that specific protocol fields appear

at known word indices.

Each packet begins with a 32-bit `len_words` field, which defines the length of the packet in words. The remaining data follows a fixed format, where important header fields are placed at predetermined offsets. For example, the EtherType field is located in Word 3 (bits [31:16]) for IPv4 identification, while the TCP flags are located in the lower byte of Word 11 and are used for detecting attack patterns.

As shown in Fig. 12, these fixed offsets allow the hardware to extract relevant fields directly as each word arrives, without requiring a search across the packet. This keeps the parsing logic simple and allows fields to be captured in a single clock cycle during streaming.

Listing 1 shows the packet representation used in the software-defined datasets. Each packet contains a length field and an array of 32-bit words, allowing packets of different sizes to be replayed through the same transmission function.

Control signals, including `TVALID` and `TLAST`, are set during this transfer to indicate valid data and the final word of a packet, respectively. This allows the hardware modules to correctly interpret packet boundaries and process incoming data in a structured manner.

In real-world network interfaces, such as Ethernet Media Access Control (MAC) layers, packet boundaries are defined by frame structures within the hardware layer [50]. The use of control signals in this system therefore simulates real-world packet streaming behaviour within the test environment.

Packet transmission is managed by the MicroBlaze processor, which reads packet data from memory and writes it to the hardware interface via memory-mapped GPIO. This allows control over packet timing and ordering, allowing simulation of different traffic scenarios, including both normal and malicious patterns.

Initial approaches for sending packet data to the system involved the use of pre-initialised memory formats (e.g. `.coe` and `.hex`). This was implemented using a Vivado Distributed Memory Generator configured as a ROM, with packet data preloaded via memory initialization files. While this approach allowed static dataset initialisation within the hardware design, it required regeneration of bitstream and re-synthesis when switching between datasets.

Additionally, the ROM introduced a one-cycle read delay, meaning that data was not available until the following clock cycle. This caused the parser to read the wrong word positions during early development, leading to incorrect extraction of protocol fields, for example TCP flags.

To address this, the design was changed to use a runtime packet replay mechanism. Packet data is defined in software and sent to the FPGA by the MicroBlaze processor, rather than being preloaded into memory. This makes it easier to switch between datasets during testing without requiring hardware reconfiguration, while also ensuring that packet data is correctly aligned during processing.

The selected packet record is transmitted to the FPGA one word at a time, as shown in Listing 2. The final word of each packet is marked using `TLAST`.

Listing 1: Example packet representation used in the C header datasets

```
static const packet_record_t syn_packet_real_world[] = {
  {
    .len_words = 15,
    .words = {
      0x90B11CA2,
      0xC0D370F3,
      ...
    }
  }
};
```

An important detail in the `send_packet` function is the 10 μs delay (`usleep(10)`) between writing each word. This delay acts as a simple synchronisation mechanism between the software and hardware domains. Since the GPIO interface does not provide hardware flow control, the delay ensures that the hardware parser has sufficient time to process each word before the next is transmitted.

Listing 2: Packet transmission using TVALID and TLAST control signals

```
static void send_packet(const packet_record_t *pkt) {
  for (int i = 0; i < pkt->len_words; i++) {
    uint32_t ctrl = CTRL_TVALID;
    if (i == pkt->len_words - 1)
```

```
      ctrl |= CTRL_TLAST;

    Xil_Out32(GPIO_PKT_BASE, pkt->words[i]);
    Xil_Out32(GPIO_CTRL_BASE, ctrl);
    usleep(10);
    Xil_Out32(GPIO_CTRL_BASE, 0);
  }
}
```

The annotated Vivado block diagram shown in Fig. 11 shows the overall architecture of the system. For clarity, key subsystems are highlighted using coloured regions and labelled accordingly.

4.3 IDS Module

The IDS module is the main hardware block responsible for processing packet data and detecting potential attacks. It operates on a streaming input, meaning that packets are processed as they arrive, without storing the entire packet.

The design is split into a number of stages, including packet parsing, feature extraction, traffic monitoring, and detection. Each stage performs a specific task and passes the results to the next stage.

This structure allows the system to analyse packets continuously with low latency. The following sections describe each part of the module in more detail.

4.3.1 Packet Parsing

The parser operates under the assumption that packets follow a standard Ethernet frame structure with IPv4 headers without optional fields. Although IPv4 headers can vary in length due to the presence of options, this design assumes a fixed header size. As a result, important protocol fields are located at fixed positions within the packet. It is worth keeping in mind, that in more general scenarios, where variable-length headers must be handled, more advanced parsing techniques may be required, such as the dynamic parsing approaches described in [51].

Packets are received as a sequence of 32-bit words via a streaming interface, with `TVALID` indicating valid data and `TLAST` marking the final word of a packet. This allows the parser to process packets sequentially without requiring full buffering of packet contents.

A word index counter is used to track the position within each packet as it is received. Based on this index, specific protocol fields are extracted at known offsets within the packet structure. This approach allows for efficient hardware implementation, as field extraction is performed using combinational logic rather than complex parsing algorithms.

For example, the EtherType field is extracted from word 3, the IP protocol from word 5, and TCP flags from word 11. By using fixed offsets, the parser can quickly identify packet types and relevant header information with minimal latency.

Table 3: Protocol fields extracted by the packet parser

Word index	Field extracted	Purpose
3	EtherType	Identifies IPv4 packets
5	IP protocol	Distinguishes TCP, UDP, and ICMP traffic
7–8	Destination IP address	Used for target-level reporting
11	TCP flags	Identifies SYN, NULL, and XMAS patterns

Listing 3: Extraction of protocol fields using word indexing

```
case (word_index)
  8'd3:  ethertype <= tdata[31:16];
  8'd5:  ip_proto  <= tdata[7:0];
  8'd7:  dst_ip[31:16] <= tdata[15:0];
  8'd8:  dst_ip[15:0]  <= tdata[31:16];
  8'd11: tcp_flags <= tdata[7:0];
endcase
```

As shown in Listing 3, protocol fields are extracted according to the current value of the `word_index` signal. The parser increments this index as each 32-bit word is received, allowing the hardware to determine which part of the packet header is currently present on `tdata`.

The EtherType field is captured at word 3 and is used to determine whether the packet contains IPv4 traffic. The IP protocol field is then captured at word 5, allowing the design to distinguish between TCP, UDP, and ICMP packets. The destination IP address spans two 32-bit words in the packet representation, so the upper and lower halves are extracted separately from words 7 and 8 before being combined into a single 32-bit value.

For TCP packets, the flags field is extracted at word 11. This value is used to identify TCP-specific behaviours, including SYN packets and flag combinations associated with NULL and XMAS scans. These extracted fields form the input to the later classification and detection stages.

This approach keeps the parsing logic simple and efficient, as the required fields can be extracted directly from known word positions without buffering the full packet.

4.3.2 Feature Extraction and Signature Detection

Using the extracted fields, the parser classifies packets based on protocol type. IPv4 packets are identified using the EtherType field, after which the IP protocol field is used to distinguish between TCP, UDP, and ICMP traffic.

For TCP packets, additional inspection of the TCP flags field allows the system to identify specific patterns such as SYN packets, as well as scan-based behaviour including NULL and XMAS packets. These classifications form the basis for subsequent detection logic.

The generated feature signals are used to update a set of hardware counters, which track the occurrence of different types of traffic. These counters provide a simple representation of network activity over time.

By monitoring these values, the system can identify abnormal traffic patterns, such as a high volume of SYN or ICMP packets, which are commonly associated with DoS attacks. These aggregated statistics are then passed to the sliding window detection module described in the following section.

4.3.2.1 Attack Signature Identification The Feature Extraction stage performs real-time pattern matching against known malicious signatures. By utilizing dedicated hardware comparators, the IDS identifies specific attack vectors at the transport layer. The primary detection signatures implemented in the RTL logic are detailed in Table 4.

Table 4: Hardware Detection Signatures and Logic Conditions

Attack Type	Protocol	TCP Flag (Hex)	Hardware Logic Condition
SYN Flood	TCP (6)	0x02	`tcp_flags == 8'h02`
NULL Scan	TCP (6)	0x00	`tcp_flags == 8'h00`
XMAS Scan	TCP (6)	0x29	`FIN (1) + PSH (1) + URG (1)`
UDP Flood	UDP (17)	N/A	`ip_proto == 8'h11`
ICMP Flood	ICMP (1)	N/A	`ip_proto == 8'h01`

The signature matching logic is described below:

- **SYN Flood Detection:** The hardware specifically monitors for packets where only the synchronization (`SYN`) bit is set. A high volume of these packets within the 250ms window indicates a half-open connection attack designed to exhaust target resources.
- **NULL Scan Detection:** This probe utilizes packets with no control bits set. Under normal TCP operation, at least one flag is expected; the presence of `0x00` is a common signature for scanners attempting to bypass stateless firewalls.
- **XMAS Scan Detection:** The system identifies this probe by the simultaneous assertion of the `FIN`, `PSH`, and `URG` flags (`0x29`). This atypical bit pattern is used by attackers for OS fingerprinting and port probing.

4.3.3 Sliding Window

The sliding window detection module is responsible for analysing network traffic over time to identify patterns associated with DoS attacks. Rather than evaluating packets individually, the system aggregates traffic statistics over a fixed time window, allowing abnormal behaviour to be detected based on overall traffic trends.

A fixed-size time window is used to group packets into discrete intervals. At the start of each window, counters are reset, and packet statistics are collected as packets are processed. Once the window completes, the aggregated values are evaluated before the counters are cleared for the next interval.

This approach allows the system to continuously monitor traffic while maintaining a consistent view of activity over time, effectively creating a sliding window of recent network behaviour.

The feature signals generated during packet parsing are used to update a set of hardware counters. These counters track metrics such as the number of TCP, UDP, and ICMP packets, as well as the occurrence of specific TCP flag patterns.

As packets are processed within a window, the corresponding counters are incremented, providing a summary of traffic behaviour over that interval. This aggregation allows the system to detect volumetric patterns that would not be visible from individual packets.

4.4 Detection and Alert Generation

At the end of each window, the accumulated counter values are compared against predefined thresholds to determine whether an attack is present. For example, a high number of SYN packets within a short time period may indicate a SYN flood, while a large volume of ICMP packets may indicate a ping-based attack.

If any of these thresholds are exceeded, the system generates a detection flag corresponding to the identified attack type. This flag is then passed to the output stage for reporting.

The sliding window mechanism is implemented entirely in hardware using a combination of counters and timing logic. A clock-driven counter is used to define the duration of each window, while separate counters track packet statistics in parallel.

At the end of each window, the current counter values are evaluated and then reset for the next interval. This ensures that the system continuously monitors traffic without requiring software intervention, enabling real-time detection with minimal latency.

A window duration of 250 ms was chosen as a balance between responsiveness to rapid traffic bursts and reducing sensitivity to short-term fluctuations in packet rates.

The volumetric detection logic is responsible for identifying attacks based on the volume of network traffic observed within each detection window. Rather than analysing individual packets in isolation, the system evaluates aggregated traffic statistics to determine whether abnormal behaviour is present.

At the end of each sliding window, the accumulated packet counts are compared against predefined thresholds. These thresholds represent expected limits for normal traffic behaviour. If the number of packets for a given protocol exceeds its corresponding threshold, the system flags a potential attack.

For example, a high number of TCP SYN packets within a short time interval may indicate a SYN flood attack, while a large volume of ICMP packets may indicate a ping flood. Similarly, elevated UDP traffic can be used to detect UDP-based flooding attacks.

The detection logic is implemented using simple comparison operations in hardware, allowing all checks to be performed in parallel with minimal latency. This ensures that attack conditions can be identified in real time without impacting overall system throughput.

The resulting detection flags are then passed to the MicroBlaze processor, where they are aggregated and reported to the user via the UART interface.

4.5 Microblaze

The MicroBlaze processor acts as the control and interface layer between the hardware detection system and the user. It is responsible for configuring the system, initiating packet transmission, and collecting detection results from the FPGA.

Packet data is stored in software as predefined datasets and transmitted to the FPGA using memory-mapped AXI GPIO interfaces. During execution, the MicroBlaze sends packet data sequentially while also controlling signals such as `TVALID` and `TLAST`.

Detection results generated by the hardware are read back by the MicroBlaze through AXI GPIO interfaces. These include packet counts, window statistics, and detection flags indicating the presence of potential attacks.

The processor aggregates this information and formats it for output via the UART interface, allowing results to be displayed on a serial terminal.

4.5.1 System Configuration

At system startup, the MicroBlaze configures the hardware detection module by writing threshold values to memory-mapped registers. The volumetric and flood thresholds are combined into a single 32-bit value and written to the `GPIO_THR_BASE` register using AXI-Lite.

This initial configuration acts as a simple handshake between software and hardware, ensuring that the detection logic operates using the correct parameters before packet processing begins.

By performing this configuration in software, the system can be easily adjusted for different traffic conditions without requiring modification of the FPGA bitstream.

4.5.2 Packet Transmission and Status Polling

Packet transmission is handled by the `send_packet` function, which writes packet data to the FPGA one word at a time. Control signals such as `TVALID` and `TLAST` are asserted to indicate valid data and the end of a packet, allowing the hardware to correctly process packet boundaries.

A short delay is introduced between writes using `usleep(10)` to ensure that the hardware has sufficient time to process each word. This is necessary as the GPIO interface does not provide a hardware backpressure mechanism.

During execution, the software continuously monitors the window ID register (`GPIO_WINID_BASE`) to detect when a new processing window has completed. When a new window is detected, the corresponding status and packet statistics are read from the hardware and stored for later reporting.

This polling-based approach allows the software to remain synchronised with the hardware without interrupting the packet processing pipeline.

4.5.3 Target Identification and Dynamic Tracking

In addition to detecting attacks, the system identifies which network node is being targeted. The hardware extracts the destination IP address from each packet and exposes it to the MicroBlaze through the `dst_ip_out` register.

The software compares this address against a predefined list of known targets, such as virtual machines and servers. For matching entries, packet statistics are updated accordingly, including counts for SYN, UDP, and ICMP traffic.

If a destination IP is not found in the predefined list, it is stored in a separate dynamic target array. This allows the system to track previously unseen addresses without requiring changes to the hardware design.

The structure used to store this per-target information is shown in Fig. 13.

Structure Field	Description
uint32_t ip	Target IP Address (hex)
const char *name	Alias: (Static: "Server1") (Dynamic: "null")
uint32_t total	Current window total (resets every 250ms)
uint32_t syn/udp/icmp	Current Window Protocol Counts
uint32_t xmas,nulls	Cumulative Scan Counts

Figure 13: Layout of the `target_stats_t` structure used for per-target traffic tracking

The `target_stats_t` structure is used by the MicroBlaze to track traffic statistics on a per-target basis. The `total` field stores the number of packets observed within the current detection window and is reset whenever a new window is detected, ensuring alignment with the hardware sliding window mechanism.

Protocol-specific counters (`syn`, `udp`, and `icmp`) are also maintained on a per-window basis, allowing software-level tracking of flood-based behaviour. In contrast, scan-based indicators (`xmas` and `nulls`) are accumulated over the entire execution, enabling the system to detect low-frequency scan activity that may not exceed thresholds within a single window.

By handling target identification in software, the hardware can remain focused on high-speed packet processing, while the MicroBlaze provides a more flexible layer for tracking and reporting network activity.

4.5.4 UART Reporting

Detection results are communicated to the user through a UART interface operating at 115,200 baud. The MicroBlaze formats the collected data into a readable output, including configuration parameters, per-window statistics, and detected attack types.

Two levels of reporting are provided. First, a window summary displays packet counts and detected activity for each processing interval. This is followed by an aggregated summary, which highlights attack types and affected targets over the full duration of the test.

Separating reporting from the hardware detection logic ensures that the high-speed packet processing pipeline is not affected by slower serial output operations.

```
====================================================
        EE695 FPGA Intrusion Detection
====================================================


=====================================
RUN CONFIGURATION
=====================================
Window Size: 250 ms
Volumetric Threshold: 20
Flood Threshold: 15


=====================================
WINDOW SUMMARY
=====================================

Window 1:
Packets in window 1: 5
Attacks in window 1:

Window 2:
Packets in window 2: 12
Attacks in window 2:

Window 3:
Packets in window 3: 12
Attacks in window 3:

Window 4:
Packets in window 4: 12
Attacks in window 4:


=====================================
Attacks Detected
=====================================
Target: VMware-VM1 (192.168.10.3)
  [!] SYN Attack (16 SYN packets)
  [!] Volumetric Attack (36 total packets)

Dynamic targets seen: 0
=====================================
              DEMO DONE
=====================================
```

Figure 14: UART output showing per-window statistics and detected attack types

An example of the system output is shown in Fig. 14. The display includes configuration parameters, per-window packet statistics, and detected attack types, demonstrating that the system is able to process packet streams and report results in real time.

4.5.5 Software Operational Logic and Temporal Synchronisation

The execution flow of the MicroBlaze control logic, shown in Fig. **??**, is designed to maintain consistent synchronisation between the software layer and the high-speed hardware detection pipeline. The system follows a structured processing loop to ensure that all packets are analysed while maintaining alignment with the hardware sliding window mechanism.

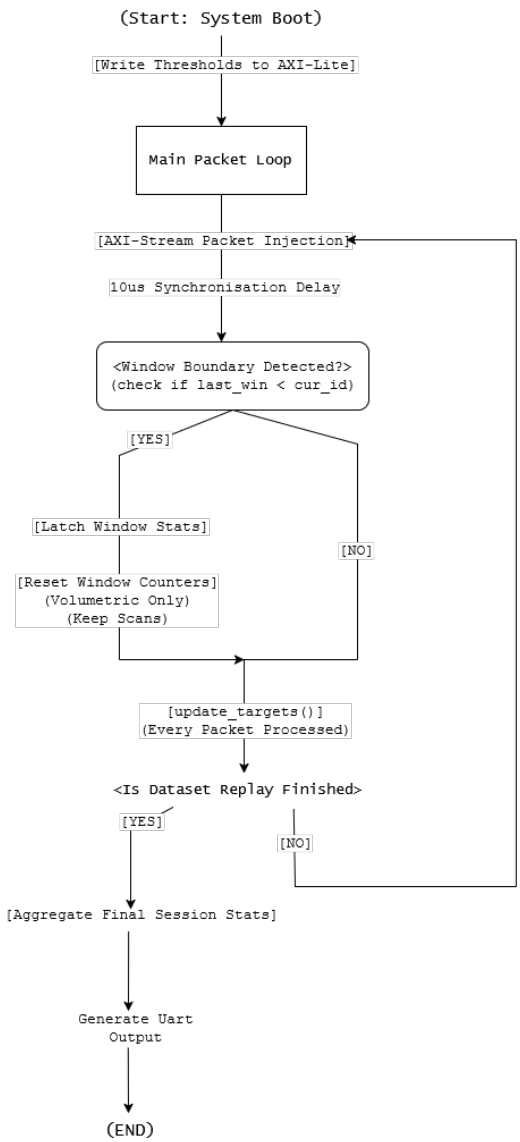


Figure 15: Software control flow of the MicroBlaze processor showing packet injection, window boundary detection, and synchronised latching of hardware statistics during real-time IDS operation

4.5.5.1 Initialisation and Hardware Configuration At system startup, the software performs a configuration step by writing detection thresholds to the hardware via the AXI-Lite interface. The volumetric and flood thresholds are combined into a single 32-bit value, as shown in Listing 4, allowing runtime adjustment of detection sensitivity without requiring FPGA reconfiguration.

Listing 4: Threshold configuration via AXI-Lite

```
uint32_t threshold_val = ((uint32_t)flood_threshold << 16) | vol_threshold;
Xil_Out32(GPIO_THR_BASE, threshold_val);
```

4.5.5.2 Packet Injection and Synchronisation Packet transmission is handled within the main processing loop. Each packet is streamed to the hardware using AXI-Stream-style control signals (`TVALID` and `TLAST`). A short delay of 10 μs is inserted between transfers to compensate for the absence of hardware backpressure (i.e. no `TREADY` signal). This ensures that the hardware parser has sufficient time to capture each word before the next is transmitted.

4.5.5.3 Window Synchronisation and Latching The hardware maintains a fixed-duration window and increments a window identifier register at each boundary. The software polls this register to detect when a new window has completed. When a transition is detected (`last_win < cur`), the software first reads the latched hardware statistics before they are overwritten.

Listing 5: Window boundary detection and statistics latching

```
while (last_win < cur && win_count < MAX_WINDOWS) {
    uint32_t stat = Xil_In32(GPIO_STAT_BASE);
    uint32_t ws   = Xil_In32(GPIO_WINSTATS_BASE);
    uint32_t packets = (ws >> 16) & 0xFFFF;

    windows[win_count].status  = stat;
    windows[win_count].packets = packets;

    last_win++;
}
```

Following this, a reset operation is applied to software-side volumetric counters, aligning them with the start of the new window. This reset-before-update ordering ensures that packets processed at the boundary are correctly attributed to the new window rather than being cleared during the reset.

4.5.5.4 Target Tracking and Reporting For every processed packet, the destination IP address extracted by the hardware is used to update per-target statistics via the `update_targets()` function. This allows the system to identify which hosts are being targeted during an attack.

Once packet replay is complete, the collected statistics are aggregated and formatted into a final report, which is transmitted via the UART interface. By performing reporting in software, the high-speed hardware detection pipeline remains unaffected by slower output operations.

4.6 Graphical User Interface

A graphical user interface (GUI) was developed to support the generation of test traffic scenarios for evaluating the IDS. Rather than manually constructing packet datasets in C, the GUI provides a structured interface for defining network traffic patterns, including source and destination nodes, attack types, and packet volumes. An example of the GUI is shown in Fig. **??**.

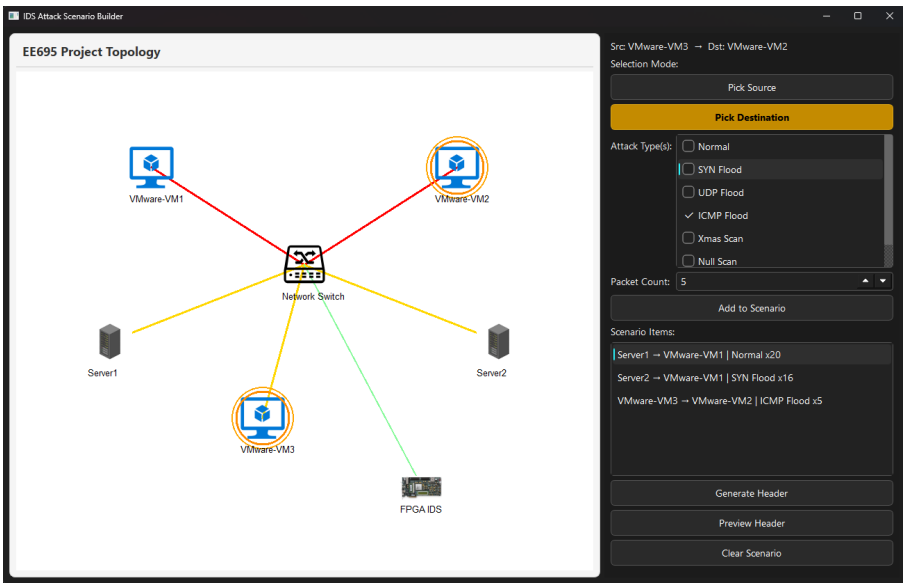


Figure 16: Graphical user interface used to generate network traffic scenarios and attack datasets for IDS evaluation

This approach improves both the speed and reliability of test generation, while ensuring that experiments can be reproduced under consistent conditions.

The interface is implemented in Python using the PySide6 framework. A topology view displays the available virtual machines, servers, network switch, and FPGA IDS node. Users can select endpoints and choose one or more traffic types, including normal traffic, SYN floods, UDP floods, ICMP floods, XMAS scans, and NULL scans. Each selected scenario is stored as a scenario item containing source and destination information, attack type, and packet count.

Once a scenario has been created, the GUI uses a packet builder module to generate a C header file containing packets in the format expected by the FPGA parser. Each packet is converted into a sequence of 32-bit words, with fields placed at the word positions used by the Verilog parser. This ensures that the generated packets are compatible with the hardware detection pipeline.

The generated header file can then be included in the Vitis application and replayed through the MicroBlaze processor during testing. This provides a faster and more consistent way to create synthetic traffic scenarios for validating the IDS.

Listing 6: Generation of parser-compatible packet words

```
def build_parser_compatible_packet(dst_ip: str, attack_type: str) -> list[int]:
    proto, flags = attack_to_proto_flags(attack_type)
    dst = ip_to_u32(dst_ip)

    words = [0x00000000] * 12
    words[3] = 0x0800 << 16
    words[5] = proto & 0xFF
    words[7] = (dst >> 16) & 0xFFFF
    words[8] = (dst & 0xFFFF) << 16
    words[11] = flags & 0xFF

    return words
```

Listing 6 shows how the GUI-generated packets are made compatible with the Verilog parser. The packet builder places key fields at the same word indices used by the hardware parser, including EtherType, protocol, destination IP address, and TCP flags.

A key requirement of the evaluation process is the ability to produce repeatable and consistent test scenarios. Manually constructing packet datasets in C is error-prone and limits the flexibility of testing. The GUI automates this process, allowing predefined attack patterns and traffic mixes to be generated in a deterministic manner.

In addition, the GUI enforces the 32-bit word-aligned format required by the hardware parser. Critical protocol fields such as EtherType and TCP flags are placed at fixed word offsets (e.g. Word 3 and Word 11), ensuring correct alignment between software-generated packets and the RTL parsing logic. This guarantees that features are extracted reliably during hardware inspection and prevents misalignment errors during testing.

4.7 Design considerations and challenges

A main challenge during development was keeping the word index aligned with the incoming packet data. When using ROM-based packet storage, the one-cycle delay in reading data meant that the parser was effectively looking at the wrong word. This led to incorrect extraction of protocol fields.

To resolve this, the design was changed to use a streaming packet replay approach, where data is sent directly from the MicroBlaze. This ensured that the data stayed aligned with the word index, allowing fields to be extracted reliably using fixed offsets.

Another issue was maintaining consistent packet alignment within the 32-bit data stream. Since packet sizes are not always naturally aligned to word boundaries, important fields such as TCP flags could shift position between packets. This was addressed by using fixed-size, word-aligned packet datasets, so that fields always appeared at the same positions.

A further challenge was the difference in speed between hardware and software. Detection signals in hardware occur over a single clock cycle, while the MicroBlaze reads values much more slowly. To avoid missing these events, counting and aggregation were moved into hardware, allowing the processor to read stable results.

4.7.0.1 Output of the parser The output of the parsing module consists of protocol indicators and packet classification signals, including identification of TCP, UDP, and ICMP traffic, as well as specific TCP flag patterns. These signals are passed to the next stage of the system, where they are used for feature extraction and detection of potential attacks.

4.7.1 Feature Extraction and Traffic Monitoring

Following packet parsing, the extracted protocol fields are used to generate higher-level features that describe the behaviour of network traffic.

The parser outputs signals indicating protocol type, such as TCP, UDP, and ICMP, as well as TCP flag information. These are used to identify patterns such as SYN packets and flag combinations associated with NULL and XMAS scans.

5 Results and Testing

A designed system must be tested to determine its effectiveness in solving the intended problem. This section presents the testing methodology used to assess the FPGA-based intrusion detection system described in Section 4, followed by an analysis of its performance under a range of traffic conditions and attack scenarios.

5.1 Testing

The FPGA-based intrusion detection system was tested to determine its effectiveness in identifying malicious network activity using both controlled and real-world conditions. The testing mainly focuses on two main aspects, per-packet classification accuracy and sliding window based attack detection.

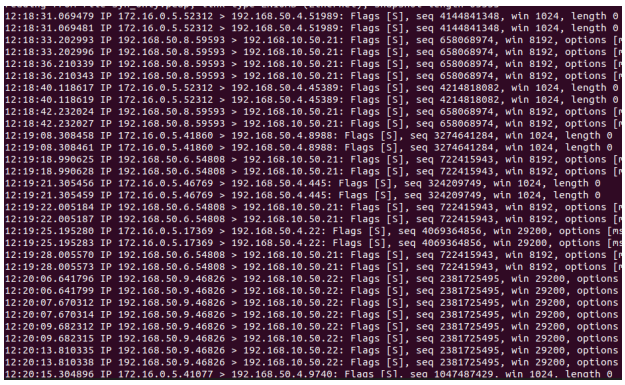
Datasets were constructed from real world PCAP (packet capture) files and then converted into a hex-based format suitable for FPGA-based testing. The packets in their original states are usable with a slight tweak to the verilog code however a hex based format was due to memnory constraints in the FPGA. Each dataset represents a specific attack type, allowing controlled evaluation of detection performance. The following sections present the methodology and results for each traffic type individually, followed by an analysis of overall system behaviour.

5.1.1 Packet Representation for a FPGA system

The initial system design intended to use the onboard Ethernet (RJ45) interface to enable real time packet capture and processing. However, implementation of this approach required access to a licensed Ethernet axi IP core, which was not available within the constraints of this project.

As a result, an alternative approach was used in which packet traces were pre recorded and replayed from on-chip memory (BRAM). This approach provided a repeatable method for evaluating the IDS, allowing testing across multiple traffic scenarios taken from real-world datasets and synthetic dataets generated for testing purposes.

While the original packets remained usable in their PCAP-derived form and could, have been processed directly by the FPGA with minor modifications to the parsing logic. However, doing so would require more exhaustive byte level parsing and additional buffering within the hardware. This would not have been feasible given the fact that the VC707 platform provides approximately 37,080 Kb Mb ($37080Kb \div 8 \approx 4.6$MB) of on chip BRAM, as seen on page 7 of the 7 Series FPGAs Data Sheet: Overview [42]. While this is enough for basic buffering and control logic, it is not sufficient to store large packet traces directly, this memory must also support buffering and control logic, the available capacity for packet storage is further reduced in practice. which is why a reduced packet format and smaller datasets were used during testing. Due to the memory limitations of the FPGA, the original PCAP packet format (shown in Fig. 17(a)) was converted into a more memory efficient representation, which is shown in Figure 17(b) below.



(a) Original PCAP packet view

```
static const packet_record_t
syn_packet_real_world[] = {
{
    .len_words = 15,
    .words = {
        0x90B11CA2,
        0xC0D370F3,
        ...
    }
}
};
```

(b) Compact FPGA packet format

Figure 17: Comparison of packet representations. The original PCAP format contains full protocol information, while the compact format stores packets as fixed 32-bit words for FPGA replay.

In standard IPv4 based communication, packets follow a structured format consisting of multiple fields distributed across several bytes. As shown in Fig. 18, the IPv4 header has a minimum size of 20 bytes, containing fields such as source and destination IP addresses and protocol identifiers.

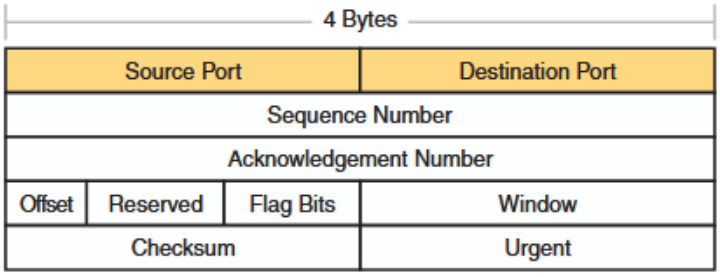


Figure 18: A Typical IPv4 TCP header

While this structure is well suited for general-purpose networking, direct processing of full PCAP byte streams on the FPGA was not adopted in this implementation due to the aforementioned memory constraints. As mentioned previously, the original system design intended to use the onboard Ethernet interface for real-time packet capture and parsing, allowing packets to be processed as they arrive rather than stored in BRAM, and therefore avoiding the associated memory constraints.

To enable better/more testing, packets were transformed into a hex representation in which the fields required for detection were kept and turned into fixed 32-bit words. Key parts of the packet such as EtherType, IP protocol, destination IP address, and TCP flags were encoded directly into predefined positions within each word. In a conventional network, these fields would be extracted by the Ethernet MAC and higher level protocol parsing logic. In this implementation the extraction step is considered already performed, allowing the FPGA to focus on the necesary fields.

5.1.2 Packet Dataset and Test Setup

To evaluate the performance of the FPGA-based intrusion detection system, network traffic datasets were obtained from publicly available intrusion detection benchmarks, specifically the CICIDS2017 and CICIDS2019 datasets. These datasets provide realistic network traffic containing both innocuous activity and also a number of attack behaviours.

Different subsets of the datasets were used to isolate specific traffic types for controlled testing. ICMP and UDP traffic were extracted from the CICIDS2019 Friday Working Hours capture, which contains a mixturse of normal background traffic and network activity representative of real-world conditions. TCP SYN packets and normal traffic were extracted from the CICIDS2017 dataset, allowing evaluation of connection based attacks such as SYN flooding then packets from both datasets were combined to create a file with multiple attack patterns.

The packet captures were processed using `tshark` and `tcpdump` to filter relevant traffic types. Custom Python scripts were then used to convert the filtered packet data into a hardware-compatible format. Each packet was represented as a sequence of 32-bit words and stored in header files, ensuring the playing of the packets through the FPGA.

For consistency across experiments, each dataset was reduced to a fixed size of 64 packets. This ensures repeatability and allows direct comparison between different traffic types and attack scenarios.

Example packets from the dataset are shown in Fig. 19. These illustrate the structure of the packet data used for classification. In particular, the TCP SYN packet (right) is characterised by the SYN flag (0x02), which is used by the IDS to identify connection initiation behaviour and detect potential SYN-based attacks.

```
    Type: IPv4 (0x0800)
Internet Protocol Version 4, Src: 192.168.10.50, Dst: 192.168.10.3
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
        0000 00.. = Differentiated Services Codepoint: Default (0)
        .... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
    Total Length: 455
    Identification: 0xc500 (50432)
    Flags: 0x40, Don't fragment
        0... .... = Reserved bit: Not set
        .1.. .... = Don't fragment: Set
        ..0. .... = More fragments: Not set
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: TCP (6)
    Header Checksum: 0xdeaa [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 192.168.10.50
    Destination Address: 192.168.10.3
Transmission Control Protocol, Src Port: 56108, Dst Port: 3268, Seq: 1, Ack: 1, Len: 403
    Source Port: 56108
    Destination Port: 3268
    [Stream index: 0]
    [Conversation completeness: Incomplete (0)]
    [TCP Segment Len: 403]
    Sequence Number: 1    (relative sequence number)
    Sequence Number (raw): 1785789788
    [Next Sequence Number: 404    (relative sequence number)]
    Acknowledgment Number: 1    (relative ack number)
    Acknowledgment number (raw): 909172649
    1000 .... = Header Length: 32 bytes (8)
    Flags: 0x018 (PSH, ACK)
        000. .... .... = Reserved: Not set
        ...0 .... .... = Nonce: Not set
        .... 0... .... = Congestion Window Reduced (CWR): Not set
        .... .0.. .... = ECN-Echo: Not set
        .... ..0. .... = Urgent: Not set
        .... ...1 .... = Acknowledgment: Set
        .... .... 1... = Push: Set
        .... .... .0.. = Reset: Not set
        .... .... ..0. = Syn: Not set
        .... .... ...0 = Fin: Not set
        [TCP Flags: ·····AP···]
    Window: 377
    [Calculated window size: 377]
```

```
    Type: IPv4 (0x0800)
Internet Protocol Version 4, Src: 192.168.10.9, Dst: 192.168.10.3
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
        0000 00.. = Differentiated Services Codepoint: Default (0)
        .... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
    Total Length: 52
    Identification: 0x000e (14)
    Flags: 0x40, Don't fragment
        0... .... = Reserved bit: Not set
        .1.. .... = Don't fragment: Set
        ..0. .... = More fragments: Not set
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 128
    Protocol: TCP (6)
    Header Checksum: 0x6559 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 192.168.10.9
    Destination Address: 192.168.10.3
Transmission Control Protocol, Src Port: 1028, Dst Port: 135, Seq: 0, Len: 0
    Source Port: 1028
    Destination Port: 135
    [Stream index: 0]
    [Conversation completeness: Incomplete (0)]
    [TCP Segment Len: 0]
    Sequence Number: 0    (relative sequence number)
    Sequence Number (raw): 3083264464
    [Next Sequence Number: 1    (relative sequence number)]
    Acknowledgment Number: 0
    Acknowledgment number (raw): 0
    1000 .... = Header Length: 32 bytes (8)
    Flags: 0x002 (SYN)
        000. .... .... = Reserved: Not set
        ...0 .... .... = Nonce: Not set
        .... 0... .... = Congestion Window Reduced (CWR): Not set
        .... .0.. .... = ECN-Echo: Not set
        .... ..0. .... = Urgent: Not set
        .... ...0 .... = Acknowledgment: Not set
        .... .... 0... = Push: Not set
        .... .... .0.. = Reset: Not set
        .... .... ..1. = Syn: Set
            [Expert Info (Chat/Sequence): Connection establish request (SYN): server port 135]
                [Connection establish request (SYN): server port 135]
                [Severity level: Chat]
                [Group: Sequence]
        .... .... ...0 = Fin: Not set
```

Figure 19: Example packets used in testing. Left: normal TCP traffic. Right: TCP SYN packet, showing the SYN flag used for attack detection.

Each packet in the dataset is associated with a known classification, including protocol type (TCP, UDP, ICMP) and, where applicable, specific flag combinations (e.g. SYN, NULL, XMAS). This enables direct comparison between expected classifications and the outputs produced by the IDS.

In addition to real-world datasets, synthetic attack traffic was generated using the Nmap (Network Mapper) tool to provide controlled examples of specific scan types. Nmap is a widely used network scanning tool that can make and transmit packets with custom TCP flag combinations, which makes it suitable for simulating attack behaviour.

Using Nmap, NULL and XMAS scan traffic were generated, a NULL scan, is where no TCP flags are set, was generated using the command:

```
sudo nmap -sN -Pn <target-ip>
```

An XMAS scan, which is where the FIN, PSH, and URG flags being set, was generated using:

```
sudo nmap -sX -Pn <target-ip>
```

Testing was carried out by replaying pre-processed packet datasets through the FPGA-based IDS. Packets were read from BRAM and transmitted sequentially into the processing pipeline, with the rate of packets being controlled using a delay between packet injections.

The system performs both per-packet classification and sliding window-based analysis. Individual packets are first classified based on protocol and flag information, after which traffic is aggregated within a fixed time window to identify higher-level behaviours such as flooding and volumetric attacks.

The following metrics were recorded during testing:

- Total number of packets processed
- Protocol and flag-based classification results
- Detection of attack types (e.g. SYN flood, ICMP flood, volumetric attack)
- Number of packets observed within each detection window

These metrics allow evaluation of both the correctness of packet classification and the behaviour of the sliding window detection mechanism under different traffic rates and threshold configurations.

5.2 Results

5.2.1 Threshold and Packet Rate Analysis

The intrusion detection system uses a time-based sliding window to detect high-volume traffic patterns. Two thresholds are defined: a volumetric threshold, which triggers when the total number of packets within a window exceeds a set value, and a flood threshold, which applies to protocol-specific traffic such as TCP SYN packets.

In the code, the thresholds were defined as:

```
uint16_t vol_threshold = 15;
uint16_t flood_threshold = 11;
```

Packet transmission rate is controlled by the delay between packet injections from BRAM, implemented using the `usleep()` function in the main program. This directly affects the number of packets that fall within each detection window. Three different rates were evaluated:

- `usleep(5000)` - increased packet rate
- `usleep(10000)` - base rate
- `usleep(20000)` - reduced packet rate

The test was performed using a dataset of normal network traffic taken from the CICIDS2019 dataset. This dataset provides realistic background traffic without significant attack patterns, making it suitable for analysing the effect of packet rate and threshold selection on false flags.

Table 5: Effect of packet transmission delay on volumetric detection

Delay	**Packets/window**	**Threshold**	**Detection result**
usleep(5000)	21, 43	Yes	Volumetric attack detected
usleep(10000)	12, 23, 22	Yes	Attack detected in Windows 2 and 3
usleep(20000)	5, 11, 12, 12, 12, 12	No	No volumetric attack detected

5.2.2 High Packet Rate: `usleep(5000)`

At a higher packet rate, a greater number of packets are processed within each 250 ms detection window, as shown in Table 5.

In this configuration, the packet count in every window exceeded the volumetric threshold, resulting in the detection of a volumetric attack. For example, window counts of 21 and 43 packets were observed, both well above the threshold of 15.

This shows that increasing the packet rate causes traffic to be concentrated within fewer windows, increasing the likelihood of threshold-based detection being triggered.

5.2.3 Baseline Packet Rate: `usleep(10000)`

At the baseline packet rate, packets are injected at a rate used in the other tests below, resulting in a more balanced distribution across detection windows when compared to the high and low packet rate configurations. The observed results are shown in Table 5.

In this configuration, the number of packets per window was observed to be 12, 23, and 22. The first window remained below the volumetric threshold of 20, while the second and third windows exceeded it. As a result, volumetric attack detection was triggered as it only needs to be in one window to trigger an attack.

This behaviour shows that the baseline rate lies close to the effective detection boundary. Unlike the high packet rate case, where all windows exceed the threshold, and the low packet rate case, where none do, the baseline rate produces mixed results. This demonstrates that small variations in packet timing can determine whether a threshold is crossed within a given window.

It is also worth noting that the total number of packets processed remains approximately constant across the different configurations, as the same dataset is replayed in each case. For the high and low packet rate tests, a total of 64 packets were observed, while the baseline configuration resulted in a slightly lower total of 57 packets within the analysed windows.

This discrepancy is attributed to the interaction between packet timing and the fixed window duration. At the baseline rate, some packets fall outside the measured window boundaries due to timing alignment, resulting in a lower observed total. This further illustrates the sensitivity of the system to packet timing, where even small changes in inter-packet delay can influence how traffic is distributed across detection windows.

5.2.4 Low Packet Rate: `usleep(20000)`

At a reduced packet rate, fewer packets are processed within each detection window, as shown in Table. 5.

In this case, packet counts per window ranged between 5 and 12, remaining below the volumetric threshold of 15. As a result, no volumetric attacks were detected, despite the same total number of packets being transmitted.

This highlights the sensitivity of the detection mechanism to packet timing. Although the overall traffic volume is unchanged, distributing packets over a longer period reduces the likelihood of exceeding the detection threshold within any given window.

5.2.5 Discussion

These results demonstrate that the detection system is strongly influenced by packet arrival rate relative to the window size. Higher packet rates concentrate traffic within a single window, increasing the likelihood of triggering threshold-based alerts, while lower rates distribute traffic across multiple windows, potentially avoiding detection.

This behaviour is expected for a sliding window-based detection mechanism and illustrates the importance of selecting appropriate threshold values and window sizes based on expected network traffic characteristics.

5.2.6 Syn Flood

Table 6: Comparison of SYN flood detection for different SYN dataset sizes

SYN Dataset Size	Observed packets per window	Detected attacks	Notes
64 packets	14, 23, 23, 4	SYN Flood; Volumetric Attack in Windows 2 and 3	Larger dataset; dynamic target tracking disabled
23 packets	14, 9	SYN Flood in Window 1 only	Reduced dataset; destination IP extracted as 192.168.50.4

The results obtained from the SYN packet dataset (Taken from CCSID 2019) show that the system is able to successfully identify SYN-based attack behaviour using both per-packet classification and sliding window analysis. Two configurations were evaluated, as shown in Table 6, a reduced dataset and a larger subset of the same dataset.

In the reduced configuration, a SYN flood was detected in the first window, where 14 packets were observed, exceeding the flood threshold of 11. Following windows contained fewer packets and did not trigger additional detections. In contrast, when a larger subset of the dataset was evaluated, multiple windows exceeded both the flood and volumetric thresholds. For example, packet counts of 23 per window resulted in simultaneous triggering of both SYN flood and volumetric attack alerts.

This behaviour is expected, as a higher concentration of SYN packets within a single detection window increases both the protocol-specific count and the total packet volume, allowing both thresholds to be exceeded. In compatison, when fewer packets are present within a window, only some windows may exceed the SYN flood threshold, and the volumetric threshold is less likely to be triggered.

Further analysis of the dataset, shown in Fig. 20, shows that the majority of SYN packets are directed towards a single destination IP address across multiple ports. This pattern is characteristic of SYN-based scanning or probing, where a host attempts to initiate connections across a range of ports. This behaviour is also reflected in the target tracking results, where only a single dynamic destination IP was identified. Although additional destination IPs are present within the dataset, they are associated with a small number of packets and do not trigger the detection window. As a result, only the main target exceeds the relevant thresholds and is recorded by the system. The observed traffic behaviour aligns with the IDS output, where repeated SYN packets within a short time interval result in the flood threshold being exceeded.

Overall, these results show that the detection system is sensitive to both traffic composition and the number of packets present within each detection window. The same underlying traffic can produce different detection outcomes depending on how packets are distributed across the fixed detection windows.

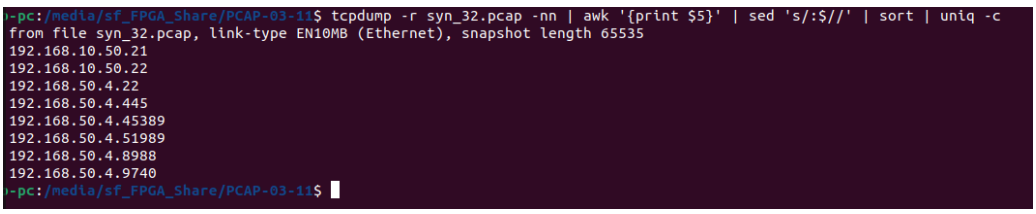


Figure 20: Extracted destination ports from the SYN packet dataset using `tcpdump`. The repeated destination IP with varying ports indicates SYN-based scanning or probing behaviour.

5.2.7 Mixed Packets

To further test the behaviour of the IDS under more realistic conditions, a mixed traffic scenario was made combining both attack and non-attack vectors. The generated scenario, shown in Fig. 21, is made up of of multiple hosts communicating through a central switch, with both normal traffic and attack traffic directed towards selected targets.

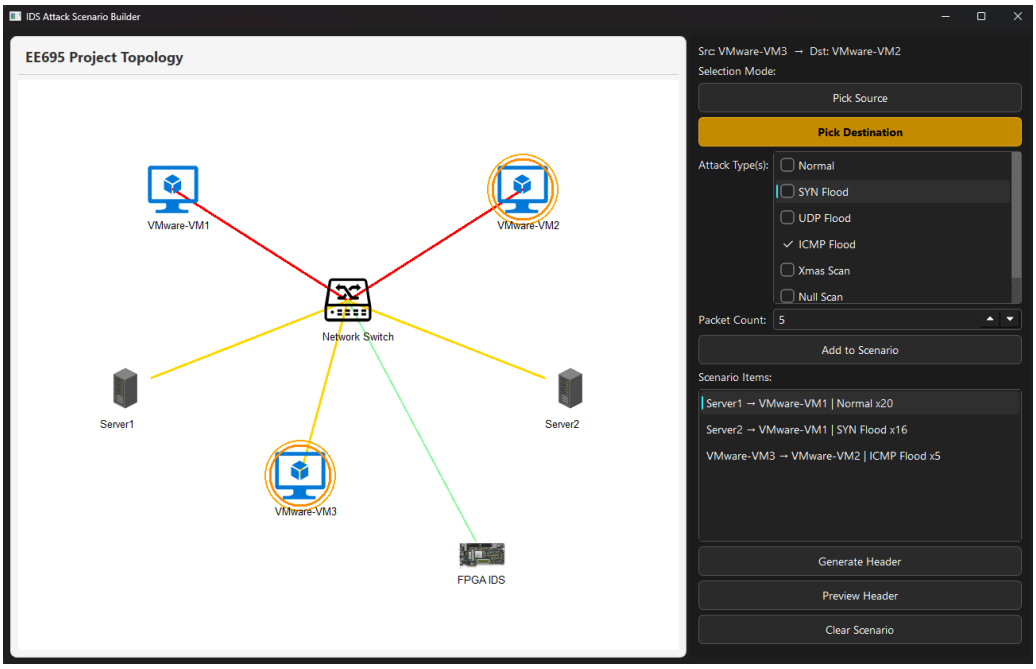


Figure 21: Mixed traffic scenario generated using the IDS attack scenario builder

In this test, three traffic sources and two destinations were defined. Server1 generated 20 normal packets directed towards VMware-VM1, representing a non attack vector. Then Server2 generated 16 TCP SYN packets towards the same destination, simulating a SYN flood attack. Additionally, VMware-VM3 generated a 5 ICMP packets directed towards VMware-VM2. It is worth noting that the source/destinatons were defined as:

Table 7: Network Node Configuration

Node Name	IP Address
Network Switch	`192.168.10.1`
VMware-VM1	`192.168.10.3`
VMware-VM2	`192.168.10.4`
Server1	`192.168.10.5`
Server2	`192.168.10.6`
VMware-VM3	`192.168.10.7`
FPGA IDS	`192.168.10.100`

This results in a total of 41 packets, with the majority of traffic concentrated on a single target. The results of this experiment are shown in Fig. 22, which is the serial monitor output coming from the FPGA. Packets were distributed across four detection windows, with counts of 5, 12, 12, and 12 packets respectively, which add up to the expected 41 packets. Three of these windows exceeded the volumetric threshold of 10 packets, and therefore a volumetric alert was triggered.

```
=====================================================
        EE695 FPGA Intrusion Detection
=====================================================


=====================================
RUN CONFIGURATION
=====================================
Window Size: 250 ms
Volumetric Threshold: 20
Flood Threshold: 15


=====================================
WINDOW SUMMARY
=====================================

Window 1:
Packets in window 1: 5
Attacks in window 1:

Window 2:
Packets in window 2: 12
Attacks in window 2:

Window 3:
Packets in window 3: 12
Attacks in window 3:

Window 4:
Packets in window 4: 12
Attacks in window 4:


=====================================
Attacks Detected
=====================================
Target: VMware-VM1 (192.168.10.3)
  [!] SYN Attack (16 SYN packets)
  [!] Volumetric Attack (36 total packets)

Dynamic targets seen: 0
=====================================
             DEMO DONE
=====================================
```

Figure 22: Window-level output for mixed traffic scenario

As we can see the IDS successfully identified a SYN flood targeting VMware-VM1. As shown in the final output, a total of 16 SYN packets were reported, exceeding the configured flood threshold and it also means the IDS system recorded all packets, as seen in Fig 21 there was 16 SYN packets generated.

In contrast, no alerts were generated for VMware-VM2. Although ICMP traffic was present, only 5 packets were observed, which remains well below the configured flood threshold. As a result, this traffic was correctly classified as non-malicious.

It is also noted that no dynamic targets were detected during this experiment. All observed destination IP addresses correspond to predefined hosts within the system, and therefore no additional target entries were created. This confirms that the dynamic target mechanism operates only on previously unseen destinations.

These results show that the IDS is capable of distinguishing between benign and malicious traffic within a mixed dataset, while also correctly associating detected attacks with specific targets. The system is able to detect protocol-specific attacks such as SYN floods even when packet arrivals are distributed across multiple detection windows, preventing individual windows from exceeding volumetric thresholds.

This behaviour shows the importance of combining per-packet classification with sliding window implementation. Sliding window analysis is effective for identifying short bursts of high traffic, whereas per-target statistics allow detection of attacks that are spread over time. This is important in mixed traffic scenarios, where malicious packets may be interwoven with normal traffic.

5.2.8 ICMP Flood

Table 8: ICMP flood detection results using CICIDS2017 dataset

Window	Packets in window	Detected attacks	Notes
Window 1	3	None	Below all thresholds
Window 2	23	ICMP Flood; Volumetric Attack	Exceeds both thresholds
Window 3	24	ICMP Flood; Volumetric Attack	Sustained high packet rate
Window 4	14	ICMP Flood	Exceeds flood threshold only

The ICMP flood detection capability of the system was tested using a dataset taken from the CICIDS2017 traffic capture. The results, as summarised in Table 8, show that the IDS is able to correctly identify ICMP-based attack behaviour.

In this experiment, packets were distributed across four detection windows containing 3, 23, 24, and 14 packets respectively. The first window remained below both the volumetric threshold of 15 and the flood threshold of 11, and therefore did not trigger any alerts. In contrast, the second and third windows exceeded both thresholds, resulting in simultaneous detection of ICMP flood and volumetric attacks.

The fourth window contained 14 packets, which is below the volumetric threshold but above the ICMP flood threshold. As a result, only the ICMP flood alert was triggered. This demonstrates that the system can distinguish between protocol-specific flooding behaviour and overall traffic volume, even when the two conditions do not occur simultaneously.

The final attack summary showed that all traffic was directed towards a single target, VMware-VM1 (192.168.10.3), with a total of 62 ICMP packets observed. This exceeds both the flood and volumetric thresholds when considered over the entire capture, resulting in both alerts being reported at the target level.

It should be noted that the IP address observed in this experiment originates from the real-world CICIDS2017 dataset. In this case, the destination address coincidentally matches one of the predefined IPs used within the test topology (192.168.10.3). As a result, the traffic is labelled as VMware-VM1 within the system, although it does not necessarily correspond to the same physical host.

These results confirm that the ICMP detection logic operates correctly on real-world traffic. Similar to the SYN flood case, detection is influenced by how packets are distributed across detection windows. High concentrations of packets within a single window result in both flood and volumetric alerts, whereas lower concentrations may only trigger protocol-specific detection.

Overall, this experiment shows that the IDS is capable of reliably detecting ICMP flood behaviour and correctly attributing it to the relevant target, even when traffic is distributed across multiple detection windows.

5.2.9 NULL Scan Detection

Table 9: NULL scan detection results using CICIDS2017 dataset

Window	Packets in window	Detected attacks	Notes
Window 1	1	None	Single packet
Window 2	24	NULL Scan; Volumetric Attack	Exceeds both thresholds
Window 3	23	NULL Scan; Volumetric Attack	Sustained high packet rate
Window 4	16	NULL Scan; Volumetric Attack	Above both thresholds

The NULL scan detection ability of the IDS was tested using a dataset derived from the CICIDS2017 traffic capture. A NULL scan is a TCP-based probing technique in which packets are transmitted with all control flags unset. Under normal TCP operation, at least one flag is expected, and therefore packets with no flags set are considered anomalous and indicative of scanning activity.

The results, summarised in Table 9, demonstrate that the system is able to correctly identify NULL scan behaviour using both per-packet classification and sliding window analysis.

In this experiment, packets were distributed across four detection windows containing 1, 24, 23, and 16 packets respectively. The first window contained only a single packet and therefore remained below the volumetric threshold of 15. However, a NULL scan alert was still triggered due to the presence of a packet with no TCP flags set, confirming that the per-packet classification logic is functioning correctly.

The remaining windows contained significantly higher packet counts, exceeding both the flood threshold of 11 and the volumetric threshold of 15. As a result, both NULL scan and volumetric attack alerts were triggered. This demonstrates that the system is capable of identifying both protocol-specific anomalies and high-volume traffic conditions within the same detection framework.

It is noted that no entries were produced in the final "Attacks Detected" section. This behaviour differs from previous experiments and is due to the software-based aggregation stage, which relies on consistent extraction of destination IP addresses. For this dataset, the destination information was not reliably captured during readback, preventing packets from being grouped into per-target statistics.

However, the window summary is generated directly by the FPGA and reflects real-time detection based on packet activity within each detection window. The correct identification of NULL scan behaviour across multiple windows therefore confirms that the hardware detection logic is operating as intended.

Overall, these results demonstrate that NULL scan detection is correctly implemented at the hardware level. While limitations in the software interface affect per-target reporting, the core detection mechanisms remain robust and capable of identifying anomalous TCP behaviour in real-world traffic.

5.2.10 XMAS Scan Detection

Table 10: XMAS scan detection results

Window	Packets in window	Detected attacks	Notes
Window 1	18	XMAS Scan; Volumetric Attack	Exceeds both thresholds
Window 2	23	XMAS Scan; Volumetric Attack	Sustained high packet rate
Window 3	23	XMAS Scan; Volumetric Attack	Consistent detection across windows

The XMAS scan detection capability of the IDS was evaluated using a dataset containing TCP packets with multiple control flags set. An XMAS scan is a type of probing technique in which packets are transmitted with several TCP flags enabled simultaneously (typically FIN, PSH, and URG), resulting in an unusual combination of flags that is not normally observed during standard TCP communication.

The results, summarised in Table 10, demonstrate that the system is able to consistently identify XMAS scan behaviour across all detection windows. In this experiment, packets were distributed across three windows containing 18, 23, and 23 packets respectively.

In all cases, the number of packets exceeded both the flood threshold of 11 and the volumetric threshold of 15. As a result, both XMAS scan and volumetric attack alerts were triggered in each window. This indicates that the IDS is able to reliably detect abnormal TCP flag combinations while also identifying high-volume traffic patterns.

No entries were produced in the "Attacks Detected" section for this experiment. As observed in the NULL scan case, this is attributed to the software-based aggregation stage, which depends on consistent extraction of destination IP addresses. For this dataset, destination information was not reliably captured, preventing packets from being grouped into per-target statistics.

Despite this limitation, the window-level results confirm that the FPGA-based detection logic is functioning correctly. The consistent identification of XMAS scan behaviour across all windows demonstrates that the system can accurately detect this class of anomaly in real-time.

Overall, these results show that the IDS is capable of reliably detecting XMAS scan activity using both per-packet classification and sliding window analysis, even when target-level aggregation is not available.

5.2.11 UDP Flood Detection

Table 11: UDP flood detection results

Window	Packets in window	Detected attacks	Notes
Window 1	18	UDP Flood; Volumetric Attack	Exceeds both thresholds
Window 2	23	UDP Flood; Volumetric Attack	Sustained high packet rate
Window 3	23	UDP Flood; Volumetric Attack	Consistent detection across windows

The UDP flood detection capability of the IDS was evaluated using a dataset containing a high volume of UDP packets directed towards a single destination. A UDP flood is a type of denial-of-service attack in which a target is overwhelmed with a large number of UDP packets, often causing resource exhaustion due to the need to process or respond to each packet.

The results, summarised in Table 11, demonstrate that the system is able to correctly identify UDP flood behaviour across all detection windows. In this experiment, packets were distributed across three windows containing 18, 23, and 23 packets respectively.

In all cases, the number of packets exceeded both the flood threshold of 11 and the volumetric threshold of 15. As a result, both UDP flood and volumetric attack alerts were triggered in each window. This indicates that the IDS is able to detect high-rate UDP traffic and correctly classify it as malicious based on packet volume and protocol type.

No entries were produced in the “Attacks Detected” section for this experiment. As observed in previous cases, this is attributed to the software-based aggregation stage, which relies on consistent extraction of destination IP addresses. For this dataset, the destination information was not reliably captured, preventing packets from being grouped into per-target statistics.

Despite this limitation, the window-level results confirm that the FPGA-based detection logic is functioning correctly. The consistent identification of UDP flood behaviour across all windows demonstrates that the system can accurately detect high-volume UDP-based attacks in real time.

Overall, these results show that the IDS is capable of reliably detecting UDP flood activity using both sliding window analysis and per-packet classification, further validating the effectiveness of the system across multiple protocol types.

5.3 System Limitations

While the proposed FPGA-based IDS demonstrates effective detection of multiple attack types, several limitations were identified during implementation and testing.

One key limitation is the reliance on on-chip BRAM for packet storage and processing. As the available BRAM on the FPGA is limited, only a relatively small number of packets can be buffered at any given time. This constrains the size of datasets that can be replayed and necessitates the use of reduced or compact packet representations. As observed during testing, larger datasets can lead to instability or incomplete processing, particularly when additional features such as dynamic target tracking are enabled.

Dynamic target tracking introduces further resource constraints. The allocation of memory for storing per-target statistics consumes valuable BRAM resources, which limits the number of concurrent targets that can be tracked. In this implementation, the number of dynamic targets is therefore restricted, and in some experiments this feature was disabled to ensure reliable system operation. This highlights a trade-off between detection capability and hardware resource usage.

Another limitation arises from the software-based aggregation stage. The final “Attacks Detected” summary depends on consistent and reliable extraction of destination IP addresses from the hardware. As observed in the NULL and XMAS scan experiments, unstable or inconsistent parsed data readback can prevent packets from being correctly grouped, resulting in missing per-target summaries despite correct detection at the window level. This indicates that the communication interface between hardware and software is a critical component of the overall system reliability.

From a detection perspective, the IDS is primarily threshold-based and focuses on a predefined set of attack signatures, including SYN floods, ICMP floods, UDP floods, and TCP flag-based scans. While effective for these cases, this approach may not generalise well to more complex or subtle attack patterns. For example, attacks such as ACK floods or low-rate distributed attacks may not be detected if they do not exceed the configured thresholds within a given window. Similarly, the system does not perform full deep packet inspection or stateful

flow analysis, as it operates primarily on packet header fields. This limits its ability to detect application-layer or evasion-based attacks.

Finally, the use of fixed detection windows introduces sensitivity to packet timing. As demonstrated in earlier experiments, the same traffic can produce different detection outcomes depending on how packets are distributed across windows. This can lead to borderline cases where attacks are inconsistently detected, particularly when traffic rates are close to threshold values.

6 Ethical Considerations

7 Conclusions and Future Work

Department of Electronic Engineering, Maynooth University

4th YEAR PROJECT RISK ASSESSMENT FORM Rev. 3

NAME AND STUDENT NUMBER:	PROJECT NAME: FPGA-based Intrusion Detection for DoS Attack
SUPERVISOR:	PROJECT LOCATION: **E.107**

BRIEF DESCRIPTION OF PROJECT

Using
Vc707 board to detect and prevent DOS/DdOS attacks

Hazards, Risk [High(H) Medium (M) Low (L)], and Control Measures

HAZARD	Risk	Controls
Data Privacy Breach	**L**	**Use anonymized or synthetic datasets for training and testing. Ensure no personally identifiable information (PII) is captured or stored. Restrict access to project data.**
System Misconfiguration	**L**	**Develop a detailed implementation plan and follow strict version control. Use a separate, isolated test network for all experiments to prevent impact on the main network.**
Hardware Damage	**L**	**Follow Electrostatic Discharge (ESD) safety procedures (e.g., use of wrist straps). Ensure correct voltage and power supply is used. Operate in a temperature-controlled environment.**
Data Loss or Corruption	**L**	**Regularly back up all project code, datasets, and configuration files to a separate, secure location (e.g., cloud storage, external drive). Use version control systems like Git.**
Eye Strain from prolonged screen use	**L**	**Ensure proper room lighting and adjust monitor brightness. Take regular breaks.**
Electrical Shock when handling the VC707 board and peripherals	**L**	**Always disconnect the power supply before connecting or disconnecting cables or components. Handle the board by its edges**
Mental Fatigue and Burnout	**L**	**Adhere to a structured work schedule with defined start and end times. Ensure you get adequate sleep and schedule time for hobbies and social activities completely unrelated to the project.**

Identified risks should be discussed with your supervisor and a safe system of work agreed. A more in-depth risk assessment may be required after initial review. Do not proceed until form is signed off.

WEEE Compliance & Recycling: Indicate below how you intend to manage and dispose of your project's material during and post project work.

N/A, project has no material that needs disposing of

Further Controls Required